

Отделение информационных технологий и вычислительной техники

КУРСОВАЯ РАБОТА

по МДК 02.01 Технология разработки программного обеспечения
специальности 09.02.07 Информационные системы и программирование

РАЗРАБОТКА МЕХАНИЗМА ПРОГРАММНОГО ДОСТУПА К МЕТАИНФОРМАЦИИ РЕЛЯЦИОННОЙ СУБД В КОДЕ КОМПЛЕКСА ДЛЯ ОРГАНИЗАЦИИ АРЕНДНОГО БИЗНЕСА В УСЛОВИЯХ МНОГОГОРОДНЕЙ СТРУКТУРЫ

Исполнитель:
студент группы № ИП-32
_____ / Голибко М.М.

Руководитель:
_____ / Прохорова И.Н.

Курсовая работа
защищена с оценкой

Новосибирск, 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	4
ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ	6
1.1. Постановка задачи	6
1.2. Обзор инструментальных средств.....	8
1.2.1. Выбор системы управления базами данных	8
1.2.2. Выбор сред разработки	9
1.2.3. Выбор дополнительных инструментов	10
1.2.4. Выбор архитектурного подхода	11
1.3. Анализ предметной области и определение возможностей системы	12
1.3.1. Анализ конкурентов	12
1.3.2. Уникальность разрабатываемого продукта.....	13
1.3.3. Определение контекста информационной системы сайта по аренде квартир	14
1.3.4. Возможности системы и варианты использования.....	17
1.4. Обоснование выбора методологии ПМИ	23
ГЛАВА 2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ	24
2.1. Анализ требований и определение спецификаций.....	24
2.2. Проектирование BPMN-диаграммы бизнес-процессов	25
2.2.1. Бизнес-процесс «Добавление квартиры в аренду».....	25
2.2.2. Бизнес-процесс «Аренда квартиры»	26
2.3. Структура базы данных (Словари данных).....	27
2.3.1. Блок «Локация и география»	27
2.3.2. Блок «Пользователи и Роли»	28
2.3.3. Блок «Объекты аренды».....	29
2.3.4. Блок «Бронирование и Оплата»	30
2.3.5. Служебные таблицы и классификаторы.....	31
2.3.6. Блок «Свойства и Ограничения».....	32
2.3.7. Блок «Контент и Избранное»	32
2.3.8. Блок «Поддержка и Статусы»	33

2.3.9. Блок «Финансы»	34
2.3.10. Блок «Управление доступом и аудит».....	35
2.3.11. Вспомогательные данные пользователей.....	35
2.3.12. Дополнительные классификаторы	36
2.3.13. Ролевые сущности (Спецификации)	37
2.3.14. Связующие таблицы ограничений	38
2.3.15. География и иерархия (Дополнение).....	39
2.3.16. Системные и ролевые связи.....	39
2.4. Разработка механизма программного доступа	39
к метainформации реляционной СУБД в коде комплекса.....	39
для организации арендного бизнеса в условиях многогородной структуры...	39
2.4.1. Реализация базового асинхронного доступа.....	41
2.4.2. Работа с метainформацией через системные функции	41
2.4.3. Формирование динамических запросов	43
2.4.4. Практический кейс: миграция данных из унаследованных систем	44
2.5. Функциональное тестирование продукта.....	45
ЗАКЛЮЧЕНИЕ	50
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	52
ПРИЛОЖЕНИЯ.....	54
ПРИЛОЖЕНИЕ 1	54
Листинг 2.1 – Класс управления доступом к данным DataDBeaver.cs.....	54
Листинг 2.2 — Библиотека интеграционных SQL-запросов	
DatabaseQueries.cs	82

ВВЕДЕНИЕ

Современный рынок арендного бизнеса характеризуется переходом от локальных сервисов к масштабным распределенным платформам, охватывающим множество городов. При расширении бизнеса в условиях многогородной структуры разработчики и интеграторы сталкиваются с проблемой вариативности данных: требования к описанию объектов недвижимости, региональные стандарты и бизнес-правила могут существенно отличаться от города к городу. В таких условиях жесткая фиксация структуры таблиц в коде приложения снижает гибкость системы и увеличивает затраты на её поддержку.

Для конечного потребителя услуг автоматизация доступа к актуальным региональным данным означает повышение прозрачности рынка аренды и сокращение времени на поиск жилья. Социальная значимость работы заключается в создании надежной цифровой среды, способствующей повышению мобильности населения и защите интересов граждан при совершении сделок с недвижимостью в разных субъектах РФ.

Необходимость создания адаптивных информационных систем, способных динамически подстраиваться под изменения структуры данных без перекомпиляции программного кода. Использование механизмов программного доступа к метаинформации реляционных СУБД (системным каталогам и схемам) позволяет реализовать «самоописываемую» архитектуру, которая критически важна для автоматизации арендного бизнеса в условиях территориальной распределенности.

Объектом исследования является программный комплекс для организации арендного бизнеса в многогородной структуре.

Предметом исследования выступает механизм программного доступа к метаинформации реляционной СУБД и его интеграция в бизнес-логику серверной части системы.

Целью работы является проектирование и разработка программного механизма на языке C#, обеспечивающего динамическое взаимодействие с метаданными СУБД PostgreSQL для эффективного управления данными аренды в условиях различных региональных сегментов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области арендного бизнеса с учетом специфики многогородной структуры.
2. Изучить теоретические основы и инструментальные средства работы с метаданными в реляционных СУБД.
3. Обосновать выбор технологического стека (C#, .NET, Npgsql) с позиции системного интегратора.
4. Реализовать программный механизм доступа к метаданным в рамках серверной логики (Backend).
5. Провести функциональное тестирование разработанного механизма.

Роль автора в проекте (интегратор): в рамках коллективной разработки автор отвечает за проектирование архитектуры хранения данных, выбор методов взаимодействия с СУБД и реализацию центрального связующего звена (DataAccess Layer), обеспечивающего доступ бизнес-логики к данным и их описанию (метаданным).

Результатом работы является программный модуль, позволяющий системе динамически получать информацию о структуре таблиц и атрибутов, что обеспечивает высокую масштабируемость комплекса при добавлении новых городов и изменении параметров объектов аренды.

ГЛАВА 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ

1.1. Постановка задачи

В рамках реализации комплексного программного решения для организации арендного бизнеса основной технической проблемой является обеспечение масштабируемости системы в условиях многогородной структуры. Разные регионы и города присутствия платформы накладывают специфические требования к описанию объектов недвижимости, методам их фильтрации и правилам обработки заявок.

Традиционный подход к разработке, предполагающий жесткое описание структуры таблиц в программном коде (статическое связывание), приводит к ряду критических проблем при расширении бизнеса:

1. Избыточность кода: необходимость создания отдельных классов или контроллеров для каждого нового типа атрибутов недвижимости, специфичных для конкретного города.
2. Сложность поддержки: любое изменение в схеме базы данных (добавление поля «наличие кондиционера» или «вид на горы») требует перекомпиляции и перезапуска всего серверного приложения.
3. Трудности интеграции: интеграция новых региональных сегментов требует ручного обновления ORM-моделей и SQL-запросов.

Целью разработки, закрепленной за ролью интегратора, является создание программного механизма, который позволит серверной части приложения динамически «узнавать» структуру хранимых данных непосредственно из СУБД. Это реализуется через программный доступ к метainформации реляционной СУБД.

В соответствии с поставленной целью, в рамках данного раздела необходимо формализовать требования к разрабатываемому механизму:

- Динамическое получение схемы данных: механизм должен уметь опрашивать системные таблицы СУБД (Information Schema) для получения списка таблиц, столбцов и типов данных, относящихся к конкретному городу или категории жилья.
- Автоматизация CRUD-операций: на основе метаданных система должна автоматически формировать SQL-запросы на чтение и запись без жесткого прописывания имен полей в коде.
- Поддержка расширяемых атрибутов: обеспечение возможности добавления новых характеристик квартир в БД без внесения изменений в скомпилированный код бэкенда.
- Унификация интерфейса доступа: создание единого API-слоя (DataAccess Layer), который будет предоставлять бизнес-логике данные вместе с их метаописанием (названия полей для фронтенда, типы валидации и т.д.).

Функциональные требования к модулю интегратора:

1. Реализация подключения к СУБД PostgreSQL с использованием драйвера Npgsql.
2. Разработка методов для чтения системного каталога (метаданных) базы данных.
3. Обеспечение маппинга (сопоставления) динамических атрибутов из БД в объекты DTO для передачи на фронтенд.
4. Интеграция разработанного механизма в общий код комплекса таким образом, чтобы фронтенд-часть могла автоматически строить формы поиска и ввода данных на основе полученной метаинформации.

Таким образом, итоговым продуктом данного этапа станет гибкое архитектурное ядро, обеспечивающее независимость программного комплекса от изменений структуры данных в условиях территориально распределенного бизнеса.

1.2. Обзор инструментальных средств

При разработке веб-сайта по аренде квартир перед командой стояла задача выбора оптимального стека технологий, обеспечивающего надёжность, производительность и удобство разработки. В данном разделе представлен анализ альтернатив и обоснование выбранных инструментов.

1.2.1. Выбор системы управления базами данных

Центральным элементом архитектуры является база данных, к которой предъявляются требования по надёжности хранения и поддержке сложных реляционных связей. Для реализации хранилища были рассмотрены наиболее популярные СУБД. Для обоснования разработки собственной системы был проведен сравнительный анализ существующих решений на рынке аренды недвижимости. Основные характеристики и недостатки систем-аналогов систематизированы в таблице 1.

Таблица 1

Сравнительный анализ систем управления базами данных

Критерий	PostgreSQL	MySQL	Microsoft SQL Server
Лицензия	Бесплатная (Open Source)	Бесплатная (Open Resource)	Платная
Поддержка JSON	Отличная	Ограниченная	Хорошая
Сложные запросы	Отличная поддержка	Хорошая	Отличная
Надёжность	Высокая	Средняя	Высокая
Кроссплатформенность	Да	Да	Ограничена

Обоснование выбора PostgreSQL:

- Бесплатная лицензия не требует дополнительных затрат
- Отличная поддержка JSON позволяет хранить гибкие структуры данных
- Поддержка сложных запросов необходима для реализации гибкого поиска по квартирам

- Надёжность и целостность данных критически важны для работы с бронированиями и платежами

Инструмент DBeaver выбран как универсальный клиент для работы с PostgreSQL благодаря бесплатности, поддержке всех необходимых функций (SQL-редактор, просмотр структуры таблиц, импорт/экспорт данных) и удобному интерфейсу.

1.2.2. Выбор сред разработки

Эффективность реализации программного механизма напрямую зависит от правильного выбора технологического стека. При подборе средств разработки учитывались такие критерии, как производительность, наличие библиотек для работы с метаданными СУБД и скорость разработки серверной части. Сравнительная характеристика и обоснование выбора используемых инструментальных средств представлены в таблице 2.

Таблица 2

Выбор сред разработки (IDE)

Инструмент	Область применения	Альтернативы	Обоснование выбора
Microsoft Visual Studio 2022	Настройка серверной логики и API (C#)	JetBrainsRider, VS Code	Глубокая интеграция с инфраструктурой Azure/Windows, средства для анализа производительности систем.
Visual Studio Code	Редактирование конфигов (YAML, JSON), скрипты автоматизации	Sublime Text, Notepad++	Огромный выбор плагинов для DevOps (Docker, Kubernetes, SSH), легкость и кроссплатформенность.

Обоснование разделения: Visual Studio обеспечивает максимальную интеграцию с экосистемой .NET, что важно для разработки на C#, в то время как VS Code идеально подходит для быстрой работы с фронтенд-кодом.

1.2.3. Выбор дополнительных инструментов

Для расширения базовых возможностей выбранной платформы и обеспечения эффективного взаимодействия с СУБД на уровне системных каталогов потребовалось использование специализированных программных библиотек. Критерии выбора и функциональное назначение дополнительных компонентов, интегрированных в проект, приведены в таблице 3.

Таблица 3

Выбор дополнительных библиотек и фреймворков

Инструмент	Область применения	Альтернативы	Обоснование выбора
ASP.NET Core	Фреймворк для backend	Node.js (Express), Django	Встроенная поддержка REST API, внедрение зависимостей, высокая производительность, встроенная документация Swagger
Npgsql	ORM для работы с PostgreSQL	Entity Framework Core, Dapper	Npgsql обеспечивает прямой доступ к системным каталогам PostgreSQL и выполнение динамических SQL-запросов. В отличие от тяжеловесных ORM, библиотека дает полный контроль над кодом и упрощает интеграцию разработанных запросов в проект.

1.2.4. Выбор архитектурного подхода

В проекте реализована трёхуровневая архитектура:

1. **Уровень доступа к данным (Data Access Layer)** – реализован интегратором в проекте DataDBeaver.c. Содержит SQL-запросы и методы для их выполнения через Npgsql.
2. **Уровень бизнес-логики (Business Logic Layer)** – реализован в сервисах (Services/) проекта Arenda.API. Содержит валидацию, обработку данных, формирование DTO.
3. **Уровень представления (Presentation Layer)** – контроллеры (Controllers/), обрабатывающие HTTP-запросы, проверяющие авторизацию и возвращающие ответы.

Такой подход обеспечивает разделение ответственности, упрощает тестирование и поддержку кода. На основании проведенного анализа и сравнения различных инструментальных средств был сформирован финальный перечень компонентов для реализации проекта. Итоговый стек технологий, использованный при разработке модулей интеграции, представлен в таблице 4.

Таблица 4

Итоговая таблица выбранного стека технологий

Категория	Выбранный инструмент
Целевая СУБД	PostgreSQL
Среда верификации запросов	DBeaver
Библиотека доступа к данным	Npgsql
Среда реализации интеграционных модулей	Microsoft Visual Studio 2022
Инструмент отладки интерфейса	Visual Studio Code

Таким образом, сформированный технологический стек (C#, .NET, PostgreSQL) обеспечивает необходимый баланс между производительностью и гибкостью разработки. Выбор низкоуровневого драйвера Npgsql является ключевым для прямой работы с метаданными системного каталога СУБД.

1.3. Анализ предметной области и определение возможностей системы

1.3.1. Анализ конкурентов

Для формирования уникального ценностного предложения и выявления конкурентных преимуществ системы необходимо провести сравнительный анализ существующих ИТ-решений: от крупных агрегаторов до сервисов полного цикла. Это позволит определить свободную нишу на рынке автоматизации аренды и обосновать необходимость разработки нового продукта.

Основные конкуренты:

- **Классические доски объявлений (Авито, Циан):** обладают огромной базой объектов и широкой аудиторией. Однако их основная модель долгое время строилась как "доска объявлений", где платформа минимально участвовала в сделке, что создавало риски мошенничества. В последнее время они внедряют онлайн-бронирование и проверку, но этот функционал часто распространяется не на все объекты.
- **Сервисы краткосрочной аренды (Airbnb, Суточно.ру):** сфокусированы на посуточной аренде и туристическом размещении. Механики долгосрочной аренды (на месяцы) проработаны слабее.
- **Управляющие сервисы полного цикла (Colife, Pik-arenda):** предлагают аренду с сервисом (all-inclusive), но часто работают только с собственным пулом квартир, выступая в роли управляющей компании, а не открытой площадки.

Таблица 5

Сравнительные преимущества разрабатываемой системы перед конкурентами

Критерий	Конкуренты (Авито, Циан)	Наше предложение
1	2	3
Модель участия	Площадка для встречи сторон	Активный гарант сделки

1	2	3
Безопасность	Частичная проверка объектов	Сквозная верификация пользователей, обязательная модерация объявлений, логирование всех действий
Автоматизация	Ручное общение в чатах	Автоматизированные бизнес-процессы (статусы заявок, уведомления, модерация)
Ролевая модель	Гость / Владелец	5 ролей (Пользователь, Арендодатель, Модератор, Админ, Поддержка)
Качество контента	Высокий риск спама и дубликатов	Предварительная модерация всех объявлений перед публикацией
Интеграция	Закрытые экосистемы	Открытый API для интеграции

Таким образом, разрабатываемый продукт предлагает сбалансированное решение: он сочетает открытость площадки (как у агрегаторов) с глубиной контроля и автоматизации сделок (как у управляющих сервисов), делая особый упор на качество контента через модерацию, надёжную бэкенд-логику и безопасность данных.

1.3.2. Уникальность разрабатываемого продукта

Современный рынок аренды жилья в России характеризуется высокой динамикой цифровизации и усилением конкуренции между платформами. Крупные игроки, такие как Циан, Авито, Яндекс.Аренда, предлагают пользователям развитые сервисы: онлайн-бронирование, проверку объектов, страхование сделок. Однако многие существующие решения либо ориентированы на краткосрочную аренду (как Airbnb), либо представляют собой

классические доски объявлений с ограниченным участием платформы в обеспечении безопасности сделок.

Уникальность разрабатываемого продукта заключается в комплексном подходе к автоматизации арендного бизнеса с акцентом на прозрачность, безопасность и удобство для всех участников процесса. В отличие от простых агрегаторов, проектируемая платформа предлагает полноценную экосистему, где каждый этап – от поиска квартиры до заключения договора и оплаты – поддерживается серверной логикой.

Ключевые уникальные характеристики:

1. **Разделение пользовательских ролей на уровне бэкенда.** Система чётко разграничивает права доступа для пяти категорий: Пользователь (арендатор), Арендодатель, Модератор, Администратор, Служба поддержки. Для каждой роли реализованы специфические сценарии (use cases), что повышает безопасность данных и исключает конфликт операций.
2. **Автоматизация бизнес-процессов.** В отличие от платформ, где коммуникация происходит в ручном режиме, разрабатываемый бэкенд автоматизирует ключевые процессы: обработку заявок на аренду, уведомление сторон об изменении статусов, модерацию объявлений.
3. **API-ориентированная архитектура.** Серверная часть проектируется как независимый модуль с открытым API, что обеспечивает гибкость интеграции с любым фронтендом (веб, мобильное приложение) и упрощает дальнейшее масштабирование системы.

1.3.3. Определение контекста информационной системы сайта по аренде квартир

Диаграмма контекста (Context Diagram) отображает разрабатываемую систему как единое целое и её взаимодействие с внешними сущностями. Система

рассматривается как «черный ящик» – показываются только её границы и то, что находится за их пределами.

Главная функция системы: «Управление процессом аренды квартир».

Внешние сущности (акторы):

1. **Пользователь (арендатор)** – лицо, ищущее квартиру для аренды.
2. **Арендодатель** – собственник жилья, размещающий объявления.
3. **Модератор** – сотрудник платформы, проверяющий объявления на соответствие правилам, принимающий решение о публикации или отклонении, обрабатывающий жалобы.
4. **Администратор** – технический специалист, управляющий системой и пользователями.
5. **Служба поддержки** – операторы, обрабатывающие обращения пользователей.
6. **Платежная система** – внешний сервис для обработки финансовых транзакций.
7. **Сторонние API-сервисы** – внешние сервисы (например, карты, погода, геолокация).

Входные данные (Inputs):

- Данные регистрации и авторизации
- Запросы на поиск квартиры
- Данные о новых объявлениях
- Запросы на бронирование
- Решения по модерации (одобрение/отклонение)
- Жалобы на объявления
- Платежная информация
- Заявки в поддержку

Выходные данные (Outputs):

- Результаты поиска (список квартир)
- Подтверждение регистрации и действий
- Уведомления (email/push)

- Решение модератора (публикация/отклонение)
- Информация для платежной системы
- Аналитические отчеты
- Ответы службы поддержки

Управление (Control):

- Законодательство РФ (о персональных данных, об аренде)
- Политика безопасности платформы
- Правила модерации (требования к объявлениям)
- Бизнес-правила (условия аренды, бронирования)

Механизмы (Mechanism):

- Серверное оборудование
- СУБД PostgreSQL
- Веб-сервер
- Разработанное ПО на ASP.NET Core

Визуальное представление взаимодействия системы с внешними сущностями и потоками данных отражено на рисунке 1.

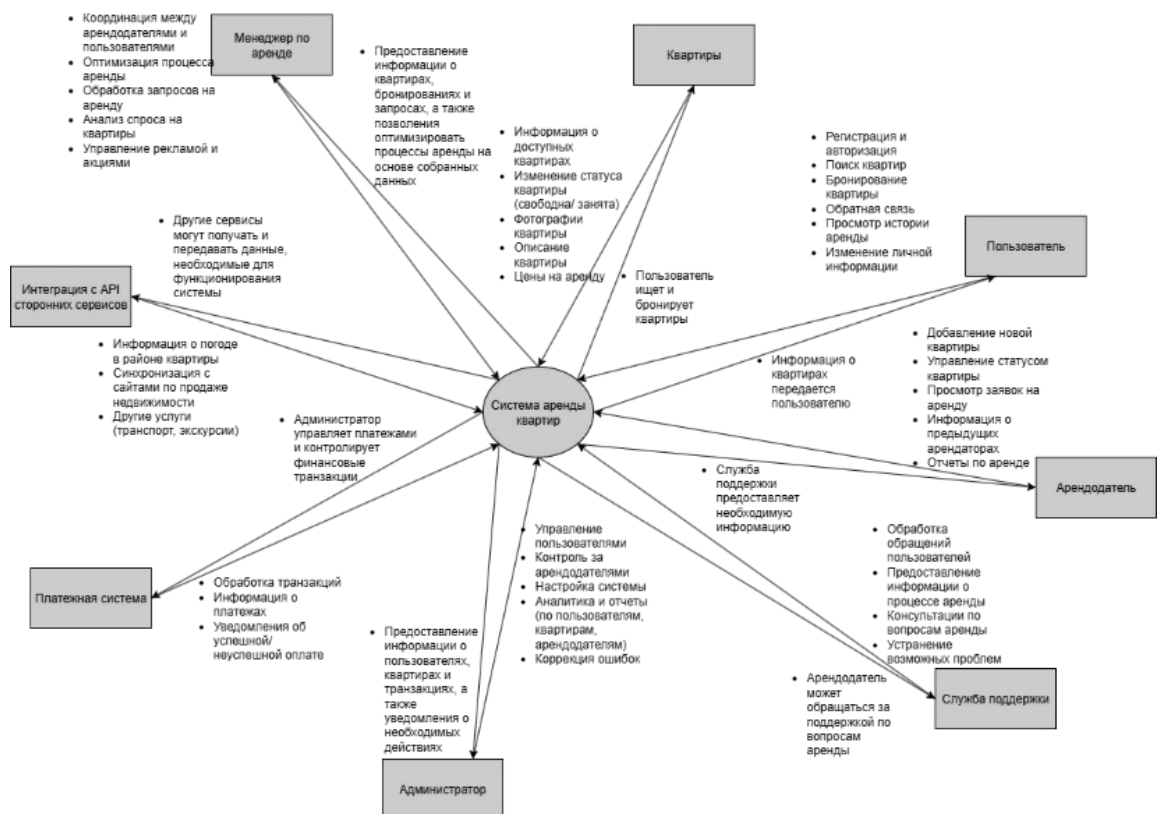


Рисунок 1 – Контекстная диаграмма системы аренды квартир

1.3.4. Возможности системы и варианты использования

Use Case (диаграмма вариантов использования) описывает функциональные требования к системе с точки зрения пользователя. Она показывает, какие действия могут выполнять различные акторы (роли) при взаимодействии с системой.

На основе анализа предметной области и выделенных ролей были разработаны диаграммы вариантов использования для каждой категории пользователей платформы аренды квартир.

Функциональные возможности системы для различных категорий пользователей детально раскрываются в диаграммах вариантов использования (Use Case). Перечень действий администратора платформы показан на рисунке 2.

Роль «Администратор»

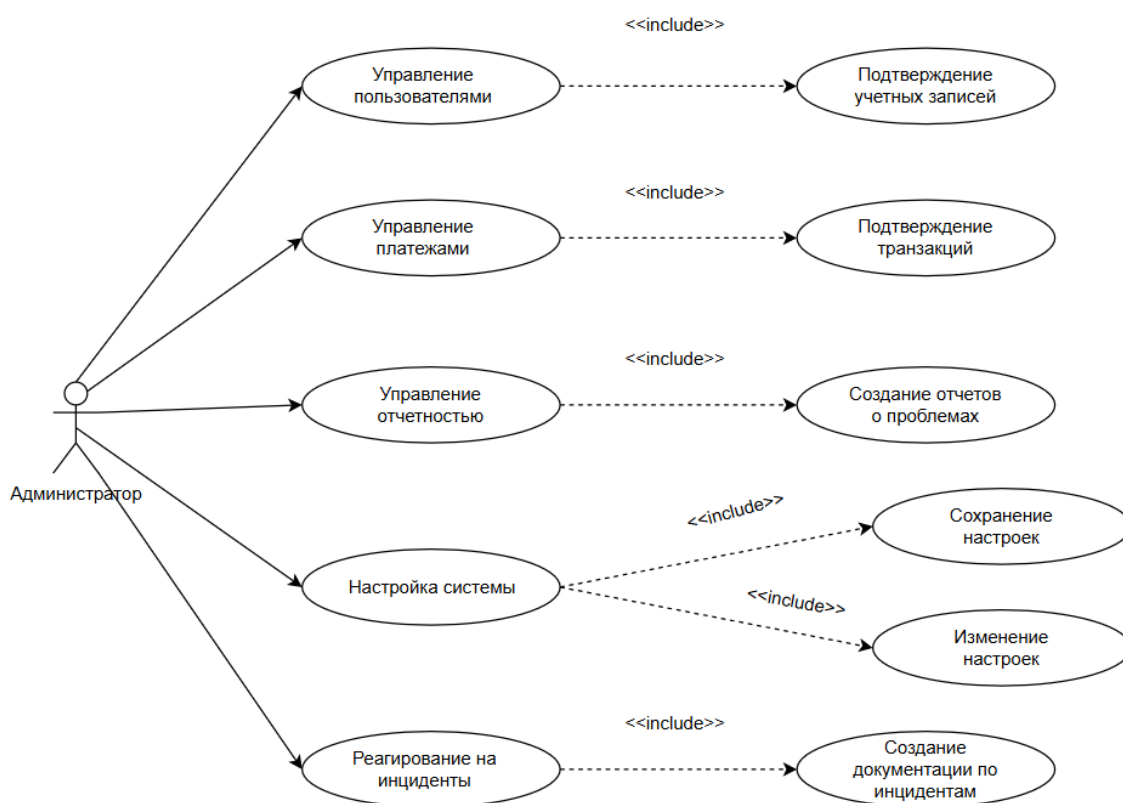


Рисунок 2 – Диаграмма Use Case для роли «Администратор»

Администратор управляет техническим состоянием платформы и правами доступа. Основные варианты использования:

- Управление пользователями – добавление, редактирование, блокировка, удаление пользователей, подтверждение учетных записей.
- Управление платежами – просмотр истории платежей, подтверждение и отмена транзакций.
- Управление отчетностью – создание отчетов о проблемах, статистике использования сайта и финансовых показателях.
- Настройка системы – изменение параметров работы платформы
- Реагирование на инциденты – обработка сбоев и нарушений безопасности, создание документации по инцидентам.

Роль «Арендодатель»

Сценарии работы собственника жилья по управлению объявлениями и заявками представлены на рисунке 3.

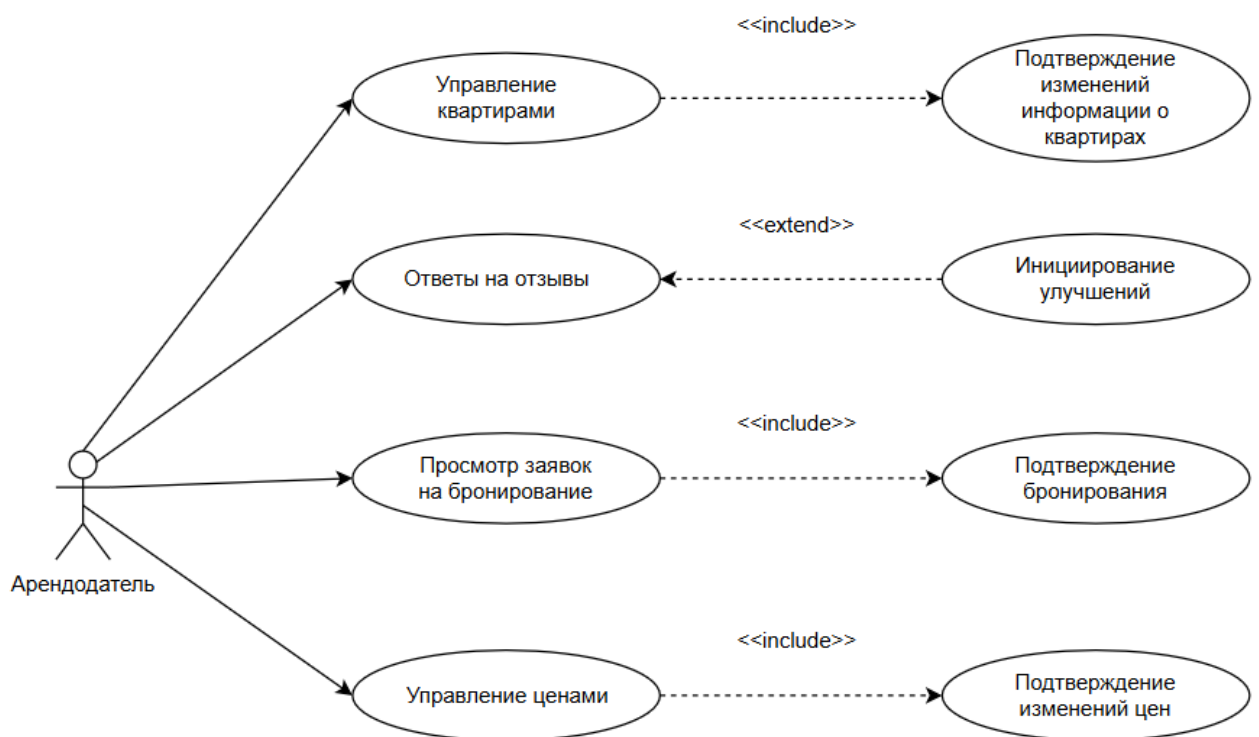


Рисунок 3 – Диаграмма Use Case для роли «Арендодатель»

Арендодатель – владелец квартиры, заинтересованный в сдаче жилья. Основные варианты использования:

- Управление квартирами – добавление, редактирование, удаление квартир (включает подтверждение изменений информации о квартирах).
- Ответы на отзывы – просмотр и ответы на отзывы (расширяется инициированием улучшений).
- Просмотр заявок на бронирование – просмотр входящих заявок (включает подтверждение или отклонение бронирования).
- Управление ценами – изменение цен на квартиры (включает подтверждение изменений цен).

Роль «Модератор»

Алгоритм действий сотрудника по контролю качества контента и модерации заявок показан на рисунке 4.

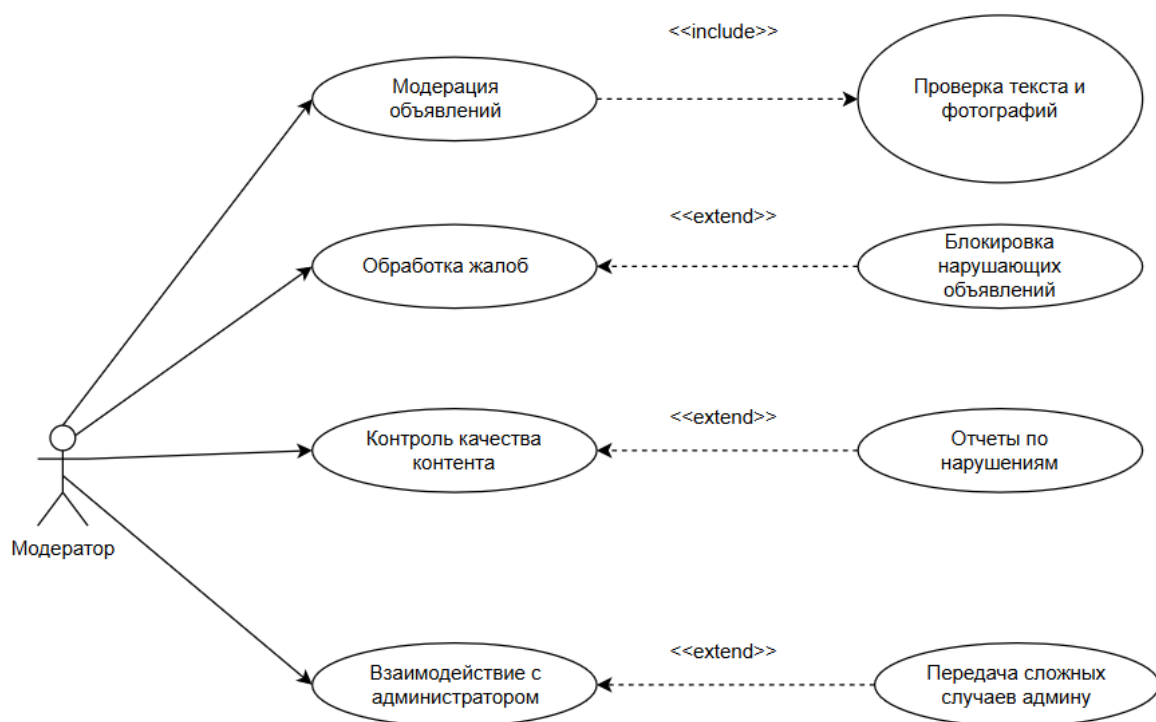


Рисунок 4 – Диаграмма Use Case для роли «Модератор»

Модератор – сотрудник платформы, ответственный за проверку контента, соблюдение правил сервиса и взаимодействие с администрацией. Основные варианты использования:

- Модерация объявлений – проверка новых и отредактированных предложений от арендодателей (включает проверку текста и фотографий на соответствие стандартам).
- Обработка жалоб – рассмотрение обращений пользователей о нарушениях правил (расширяется блокировкой нарушающих объявлений).
- Контроль качества контента – регулярный мониторинг базы данных на предмет актуальности и достоверности (расширяется составлением отчетов по нарушениям).
- Взаимодействие с администратором – координация действий с руководством системы (расширяется передачей сложных случаев админу для окончательного решения).

Роль «Пользователь»

Основные варианты использования системы со стороны клиента (арендатора) приведены на рисунке 5.

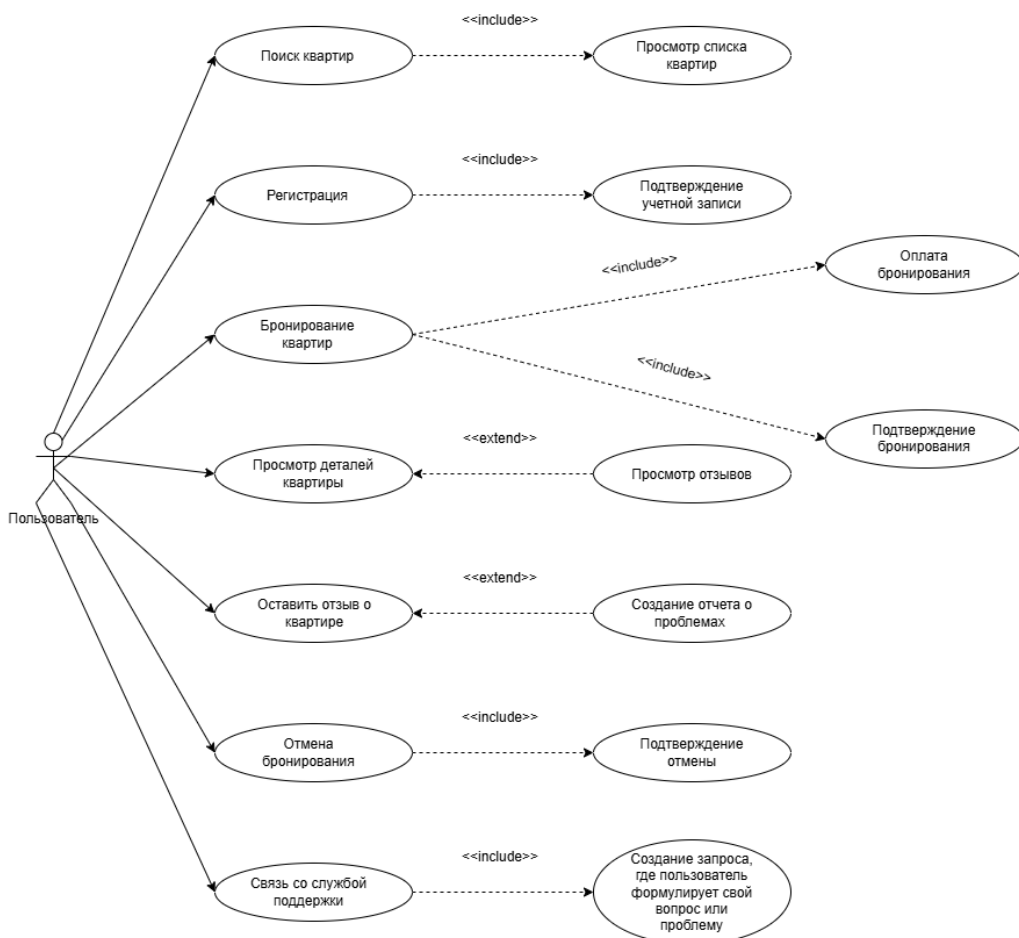


Рисунок 5 – Диаграмма Use Case для роли «Пользователь»

Пользователь – лицо, заинтересованное в поиске и аренде жилья через платформу. Основные варианты использования:

- Поиск квартир – подбор вариантов по заданным критериям (включает просмотр списка квартир).
- Регистрация – создание личного профиля в системе (включает подтверждение учетной записи).
- Бронирование квартир – оформление заявки на аренду объекта (включает оплату бронирования и подтверждение бронирования).
- Просмотр деталей квартиры – ознакомление с подробным описанием и характеристиками объекта (расширяется просмотром отзывов других пользователей).
- Оставить отзыв о квартире – публикация мнения о качестве проживания (расширяется созданием отчета о проблемах).
- Отмена бронирования – процедура отказа от аренды (включает подтверждение отмены).
- Связь со службой поддержки – обращение за помощью (включает создание запроса, где пользователь формулирует свой вопрос или проблему).

Роль «Служба поддержки»

Процессы обработки инцидентов и консультирования пользователей сотрудниками поддержки отражены на рисунке 6.

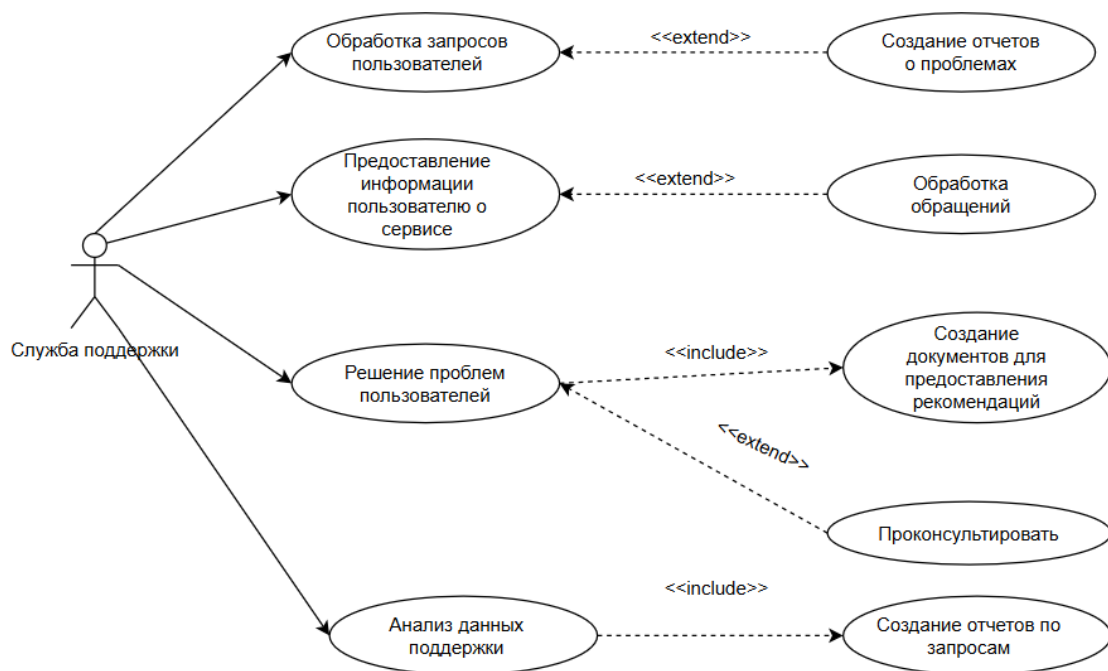


Рисунок 6 – Диаграмма Use Case для роли «Служба поддержки»

Служба поддержки – подразделение, отвечающее за помощь пользователям, консультирование и анализ качества работы сервиса. Основные варианты использования:

- Обработка запросов пользователей – прием и классификация входящих сообщений (расширяется созданием отчетов о проблемах).
- Предоставление информации пользователю о сервисе – информирование о правилах и возможностях платформы (расширяется обработкой обращений).
- Решение проблем пользователей – устранение возникших трудностей (включает создание документов для предоставления рекомендаций и расширяется возможностью проконсультировать пользователя).
- Анализ данных поддержки – сбор и изучение статистики по тикетам (включает создание отчетов по запросам).

1.4. Обоснование выбора методологии ПМИ

В рамках данной работы для верификации разработанного механизма выбрана стратегия тестирования методом «черного ящика». Данный подход заложен в основу Программы и методики испытаний (ПМИ) и обоснован ролью автора как системного интегратора: основной задачей является проверка корректности функционирования комплекса как единого целого при взаимодействии с многогородной структурой данных.

Испытания направлены на проверку внешних проявлений работы механизма:

1. Соответствие возвращаемых данных (структуры фильтров, списков квартир) запросам пользователя.
2. Корректность отработки бизнес-сценариев в условиях регионального распределения (логическая изоляция данных разных регионов в рамках единой схемы).
3. Соблюдение требований к корректности отклика и стабильности соединения.

Внутренняя логика программных методов и модульное тестирование («белый ящик») вынесены за рамки данной части ПМИ.

В ходе выполнения первой главы был проведен комплексный анализ предметной области арендного бизнеса. Сравнительный анализ существующих решений (Авито, Циан, Airbnb) позволил выявить нишу для продукта, сочетающего открытость площадки с автоматизацией на уровне серверной логики. На основе поставленных задач был обоснован выбор технологического стека: языка C# (.NET), СУБД PostgreSQL и драйвера Npgsql для прямого взаимодействия с данными. Спроектированная архитектура и ролевая модель доступа заложили фундамент для реализации гибкого механизма управления метаданной, позволяющего системе адаптироваться к региональной специфике без изменения программного кода.

ГЛАВА 2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ

2.1. Анализ требований и определение спецификаций

При реализации механизма доступа к метайнформации были определены следующие требования к структуре хранения данных доступа к метайнформации, специфичные для многогородной структуры арендного бизнеса:

1. Динамическая типизация атрибутов: система должна позволять добавлять специфические для города параметры (например, «наличие кондиционера» для южных регионов) без изменения схемы классов в C#.
2. Целостность метаданных при миграциях: механизм должен автоматически синхронизировать состояние последовательностей (sequences) при вставке данных, чтобы избежать конфликтов идентификаторов в распределенной базе.
3. Унификация доступа через Npgsql: использование низкоуровневого драйвера вместо тяжелых ORM для минимизации задержек при обращении к системным каталогам СУБД.
4. Асинхронность: все операции по извлечению метаданных и данных должны выполняться асинхронно (async/await) для обеспечения высокой отзывчивости интерфейса.

Спецификация приемо-сдаточных испытаний (ПМИ) методом «черного ящика» включает в себя верификацию следующих показателей:

- **Корректность CRUD-операций:** сохранение и чтение данных через динамические SQL-запросы.
- **Целостность связей:** отсутствие ошибок при автоматической синхронизации последовательностей (sequences) после массового импорта объявлений.
- **Отказоустойчивость:** корректная выдача сообщения об ошибке при попытке доступа к метаданным несуществующей таблицы или города.

Таким образом, сформированный стек технологий (C#, PostgreSQL, Npgsql) обеспечивает необходимый баланс между скоростью разработки и полным контролем над выполнением запросов к метаданным. Это закладывает фундамент для реализации гибкого Data Access Layer.

2.2. Проектирование BPMN-диаграммы бизнес-процессов

Для формализации логики работы проектируемой системы аренды жилья и визуализации взаимодействия между пользователями и программными модулями было выполнено моделирование ключевых бизнес-процессов в нотации BPMN 2.0.

2.2.1. Бизнес-процесс «Добавление квартиры в аренду»

Процесс размещения объявления регламентирует последовательность действий арендодателя по вводу данных и их последующую проверку модератором. Графическая схема данного процесса представлена на рисунке 7.

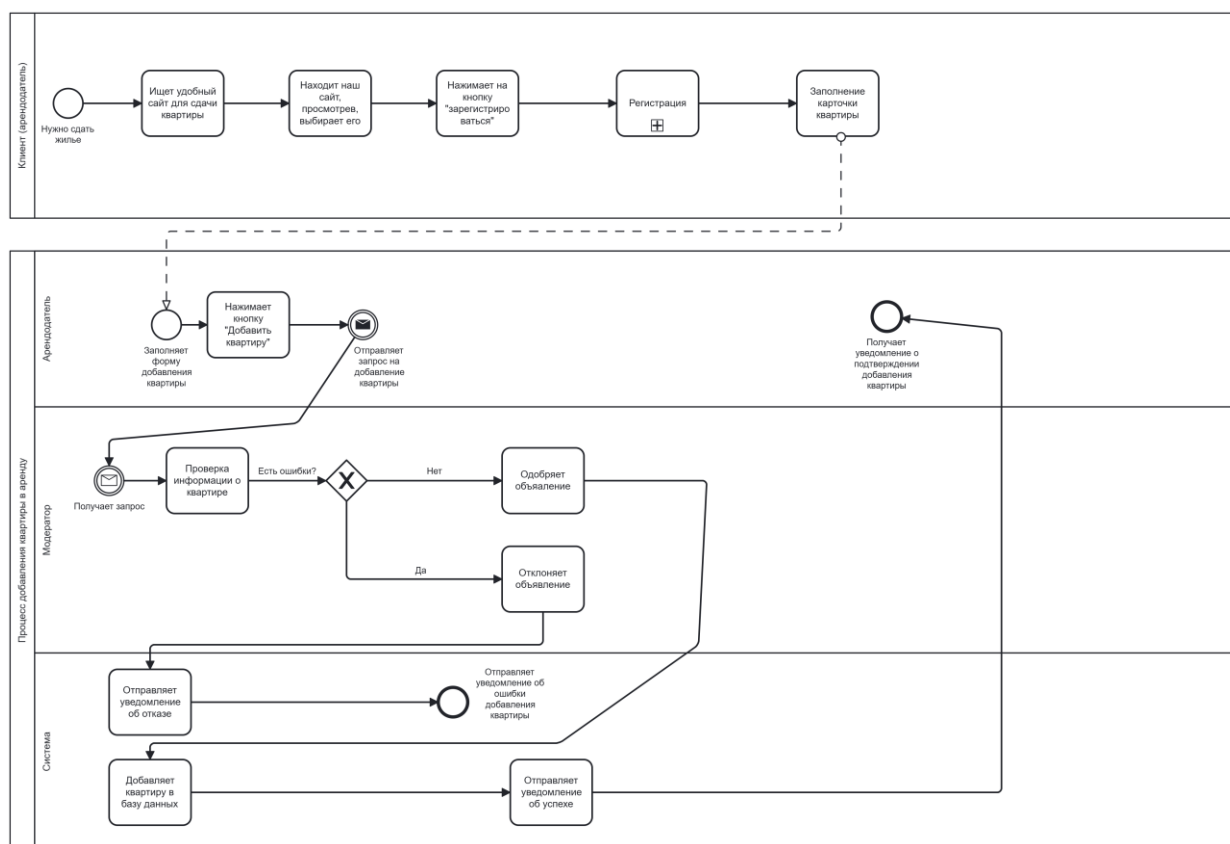


Рисунок 7 – Диаграмма бизнес-процесса «Добавление квартиры в аренду»

Как показано на рисунке 7, в процессе выделено четыре дорожки: «Клиент (арендодатель)», «Арендодатель» (интерфейсный уровень), «Модератор» и «Система».

Этап инициации: процесс запускается при возникновении у клиента потребности сдать жилье. После прохождения регистрации и авторизации пользователь приступает к заполнению характеристик объекта.

Модерация: данные поступают модератору для проверки. С помощью исключающего шлюза принимается решение о публикации. При наличии ошибок система уведомляет пользователя об отказе, при их отсутствии – подтверждает размещение.

Завершение: система фиксирует данные в БД и переводит объект в статус «Опубликовано».

2.2.2. Бизнес-процесс «Аренда квартиры»

Процесс бронирования описывает алгоритм взаимодействия арендатора с платформой и владельцем недвижимости. Схема процесса приведена на рисунке 8.

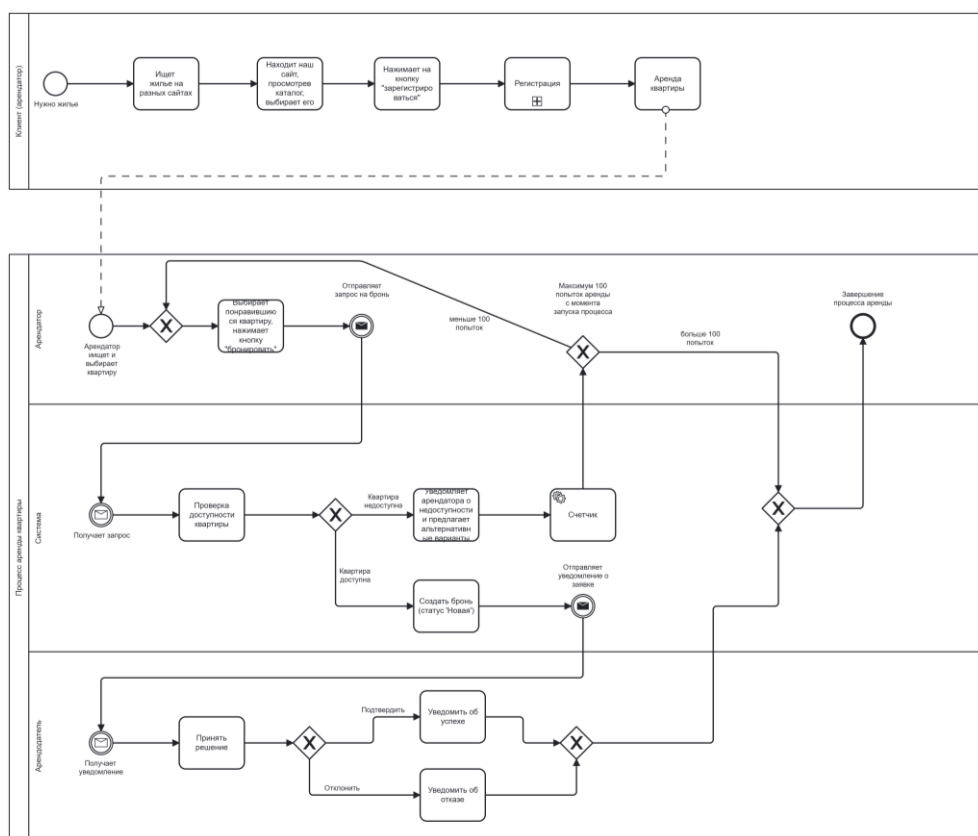


Рисунок 8 – Диаграмма бизнес-процесса «Аренда квартиры»

Согласно диаграмме на рисунке 8, процесс включает следующие ключевые этапы:

- Поиск и выбор: арендатор осуществляет поиск по фильтрам и инициирует запрос на бронирование выбранного объекта.
- Проверка доступности: система в автоматическом режиме проверяет занятость дат. Если квартира занята, запускается цикл подбора альтернатив (с ограничением до 100 итераций). Если свободна – создается бронь со статусом «Новая».
- Принятие решения: владелец квартиры вручную подтверждает или отклоняет заявку.
- Финализация: вне зависимости от исхода, система генерирует уведомление для арендатора, информируя его о статусе бронирования.

2.3. Структура базы данных (Словари данных)

В данном разделе представлена физическая структура базы данных, реализованная в СУБД PostgreSQL. Описание приведено в формате словарей данных, сгруппированных по функциональным блокам.

2.3.1. Блок «Локация и география»

Этот блок обеспечивает иерархическую привязку объектов аренды к территории. Структура таблиц для хранения стран, регионов и городов описана в таблицах 6, 7 и 8 соответственно.

Таблица 6

Словарь данных таблицы strana (Страны)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор страны
nazvanie_strany	VARCHAR(100) (U)	Уникальное название страны

Для обеспечения корректной иерархии и связи «страна — регион» используется таблица областей, структура которой приведена в таблице 7.

Таблица 7

Словарь данных таблицы region (Регионы и области)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор региона
strana_id	INTEGER(10) (FK)	Ссылка на страну
nazvanie_regiona	VARCHAR(100)	Название субъекта

Завершает блок географической привязки описание атрибутов таблицы городов, представленное в таблице 8.

Таблица 8

Словарь данных таблицы gorod (Города)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор города
region_id	INTEGER(10) (FK)	Ссылка на регион
nazvanie_goroda	VARCHAR(100)	Название города

2.3.2. Блок «Пользователи и Роли»

Атрибутивный состав сущностей для управления учетными записями и правами доступа представлен в таблицах 9 и 10.

Таблица 9

Словарь данных таблицы polzovatel (Пользователи)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор пользователя
login	VARCHAR(50) (U)	Уникальный логин
parol	VARCHAR(255)	Хэш пароля
email	VARCHAR(50) (U)	Адрес электронной почты
telefon	VARCHAR(20) (U)	Контактный телефон
imya	VARCHAR(50)	Имя пользователя
familiya	VARCHAR(50)	Фамилия пользователя
data_rozhdeniya	DATE(8) (N)	Дата рождения

Для реализации ролевой модели доступа и разграничения полномочий между категориями пользователей используется справочник, структура которого приведена в таблице 10.

Таблица 10

Словарь данных таблицы tip_rolі (Типы ролей)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор роли
nazvanіe_rolі	VARCHAR(50) (U)	Название (Админ, Модератор и т.д.)
opisanіe	VARCHAR(255) (N)	Описание прав доступа

2.3.3. Блок «Объекты аренды»

Центральная сущность системы, содержащая подробные характеристики объектов недвижимости, описана в таблице 11.

Таблица 11

Словарь данных таблицы kartochka_kvartiry (Карточки квартир)

Имя столбца	Тип данных	Описание
1	2	3
id	INTEGER(10) (PK)	Идентификатор карточки квартиры
arendodatel_id	INTEGER(10) (FK)	Ссылка на арендодателя
gorod_id	INTEGER(10) (FK)	Ссылка на город
opisanіye_kvartiry	VARCHAR(1000)	Текстовое описание объекта
adres_kvartiry	VARCHAR(255) (U)	Уникальный адрес квартиры
ploschad	INTEGER(10)	Площадь объекта в кв. м
tsena	DECIMAL(10, 2)	Стоимость аренды
komnat	INTEGER(10)	Количество комнат
etazh	INTEGER(10)	Этаж
etazhnost	INTEGER(10)	Всего этажей в здании
remont	VARCHAR(50)	Тип отделки/ремонта
mebel	SMALLINT(2)	Наличие мебели (0/1)
tehnika	SMALLINT(2)	Наличие бытовой техники (0/1)
internet	SMALLINT(2)	Доступ к интернету (0/1)

1	2	3
parkovka	SMALLINT(2)	Наличие парковочного места (0/1)
lift	SMALLINT(2)	Наличие лифта в доме (0/1)
balkon	SMALLINT(2)	Наличие балкона/лоджии (0/1)
zhivotnye_dopustimy	SMALLINT(2)	Разрешено ли проживание с животными
deti_dopustimy	SMALLINT(2)	Разрешено ли проживание с детьми
kuriye_dopustimo	SMALLINT(2)	Разрешено ли курение
min_srok_v_sutkah	INTEGER(10)	Минимальный срок аренды (сутки)
zalog	DECIMAL(10, 2)	Размер страхового залога

2.3.4. Блок «Бронирование и Оплата»

Параметры хранения данных о фактах аренды и проведенных платежах зафиксированы в таблицах 12 и 13.

Таблица 12

Словарь данных таблицы bronirovaniye (Бронирования)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор брони
arendator_polzovatel_id	INTEGER(10) (FK)	Ссылка на пользователя-арендатора
kartochka_kvartiry_id	INTEGER(10) (FK)	Ссылка на объект аренды
status_bronirovaniya_id	INTEGER(10) (FK)	Ссылка на текущий статус
data_zayezda	DATE	Дата начала аренды
data_vyyezda	DATE	Дата завершения аренды

Для каждой записи о бронировании формируется финансовый документ. Структура полей для учета платежных транзакций и предоплат представлена в таблице 13.

Таблица 13

Словарь данных таблицы predoplata (Платежи)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор транзакции
bronirovaniye_id	INTEGER(10) (FK)	Ссылка на бронирование
status_predoplati_id	INTEGER(10) (FK)	Ссылка на состояние оплаты
sposob_oplaty_id	INTEGER(10) (FK)	Ссылка на метод оплаты
summa	DECIMAL(10, 2)	Сумма к оплате
data_platezha	TIMESTAMP	Дата и время проведения оплаты
nomer_transaktsii	VARCHAR(100) (N)	Номер транзакции

2.3.5. Служебные таблицы и классификаторы

Для хранения обратной связи от пользователей и фиксации результатов проверки контента модераторами используется структура, представленная в таблице 14.

Таблица 14

Словарь данных таблицы otzyvy (Отзывы)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор отзыва
moderator_polzovatel_id	INTEGER(10) (FK)	Ссылка на модератора (проверка)
bronirovaniye_id	INTEGER(10) (FK)	Ссылка на факт аренды
reyting_klassifikator_id	INTEGER(10) (FK)	Оценка по шкале 1–5
tekst_otzyva	VARCHAR(1000)	Текстовый комментарий клиента
date	TIMESTAMP	Дата и время написания
odobreno	BOOLEAN/BIT	Флаг прохождения модерации

2.3.6. Блок «Свойства и Ограничения»

Состав справочников и связующих таблиц для учета особых условий аренды приведен в таблицах 15, 16 и 17.

Таблица 15

Словарь данных таблицы `nalichiye_detey` (Справочник: Наличие детей)

Имя столбца	Тип данных	Описание
<code>id</code>	INTEGER(10) (PK)	Идентификатор возрастного ценза
<code>nalichiye</code>	VARCHAR(100)	Название категории (например, «Младше 3 лет»)
<code>opisanie</code>	VARCHAR(255) (N)	Дополнительные пояснения

Параметры классификации объектов по типам предоставляемых удобств описаны в таблице 16.

Таблица 16

Словарь данных таблицы `nalichiye_zhivotnykh` (Справочник: Животные)

Имя столбца	Тип данных	Описание
<code>id</code>	INTEGER(10) (PK)	Идентификатор типа животных
<code>nazvanie</code>	VARCHAR(50)	Категория (собака, кошка, птицы)

Связующая сущность, объединяющая конкретные объекты аренды с выбранными услугами и ограничениями, представлена в таблице 17.

Таблица 17

Словарь данных таблицы `arendator_nalichiye_detey` (Связь: Арендаторы и дети)

Имя столбца	Тип данных	Описание
1	2	3
<code>arendator_polzovatel_id</code>	INTEGER(10) (FK)	Ссылка на профиль арендатора
<code>nalichiye_detey_id</code>	INTEGER(10) (FK)	Ссылка на возрастную категорию

2.3.7. Блок «Контент и Избранное»

Структура данных для медиаконтента и персональных списков избранного описана в таблицах 18 и 19.

Таблица 18

Словарь данных таблицы photo_kvartiry (Фотографии объектов)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор изображения
kartochka_kvartiry_id	INTEGER(10) (FK)	Ссылка на объект аренды
url_photo	VARCHAR(500)	Путь к файлу изображения
porjadok	INTEGER(3)	Порядок отображения в галерее

Для обеспечения возможности сохранения пользователями понравившихся объектов используется таблица избранного, структура которой приведена в таблице 19.

Таблица 19

Словарь данных таблицы izbrannoe (Избранное)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор записи
arendator_polzovatel_id	INTEGER(10) (FK) (U)	Пользователь, добавивший в избранное
kartochka_kvartiry_id	INTEGER(10) (FK) (U)	Выбранный объект жилья
data_dobavleniya	TIMESTAMP	Дата и время сохранения

2.3.8. Блок «Поддержка и Статусы»

Логика регистрации обращений в поддержку и статусов объектов представлена в таблицах 20 и 21.

Таблица 20

Словарь данных таблицы zaprosi_podderzhki (Запросы в поддержку)

Имя столбца	Тип данных	Описание
1	2	3
id	INTEGER(10) (PK)	Идентификатор запроса
sluzhba_podderzhki_id	INTEGER(10) (FK)	Ссылка на сотрудника службы поддержки
bronirovaniye_id	INTEGER(10) (FK) (N)	Ссылка на бронирование

1	2	3
kartochka_kvartiry_id	INTEGER(10) (FK)	Ссылка на объект недвижимости (квартиру)
polzovatel_id	INTEGER(10) (FK)	Ссылка на пользователя (клиента)
opisanie_problemy	VARCHAR(500)	Описание возникшей проблемы
data_podachi	DATE	Дата подачи обращения
data_resheniya	DATE (N)	Дата решения вопроса
kommentarii	VARCHAR(1000) (N)	Дополнительные комментарии по решению

Для отслеживания жизненного цикла заявок на аренду используется классификатор состояний, структура которого представлена в таблице 21.

Таблица 21

Словарь данных таблицы status_bronirovaniya (Статусы бронирования)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор статуса
nazvanie	VARCHAR(50) (U)	Наименование (Новая, Подтверждена и пр.)

2.3.9. Блок «Финансы»

Классификатор доступных методов оплаты приведен в таблице 22.

Таблица 22

Словарь данных таблицы sposob_oplaty (Способы оплаты)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор метода оплаты
nazvanie	VARCHAR(50) (U)	Тип (Карта, Перевод, Наличные)

2.3.10. Блок «Управление доступом и аудит»

Служит для регистрации таблиц и атрибутов, к которым настраиваются права. Перечень объектов и матрица доступа к ним описаны в таблицах 23 и 24.

Таблица 23

Словарь данных таблицы object_bd (Объекты базы данных)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор объекта системы
nazvanie_tablitsy	VARCHAR(255)	Системное имя таблицы
nazvanie_atributa	VARCHAR(255)	Имя конкретного столбца (если применимо)

Определяет права доступа конкретных ролей к объектам БД.

Матрица распределения прав доступа для различных категорий пользователей к объектам системы представлена в таблице 24.

Таблица 24

Словарь данных таблицы roli_obojekty (Связь ролей и объектов)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Технический ID записи
tip_rol_i_id	INTEGER(10) (FK)	Ссылка на роль (админ, модератор и т.д.)
object_bd_id	INTEGER(10) (FK)	Ссылка на объект (таблицу/атрибут)

2.3.11. Вспомогательные данные пользователей

Сведения о вспомогательных данных профилей и сотрудниках поддержки содержатся в таблицах 25 и 26.

Таблица 25

Словарь данных таблицы avatary_polzovatelya (Аватары)

Имя столбца	Тип данных	Описание
1	2	3
id	INTEGER(10) (PK)	Идентификатор изображения
polzovatel_id	INTEGER(10) (FK)	Ссылка на владельца аватара

1	2	3
url_avatar	VARCHAR(500) (N)	Ссылка на файл изображения профиля
imya_faila	VARCHAR(200) (N)	Оригинальное имя загруженного файла
data_obnovleniya	TIMESTAMP	Дата последней смены аватара

Для обеспечения оперативной связи с клиентами и обработки инцидентов используется реестр сотрудников службы поддержки, структура которого представлена в таблице 26.

Таблица 26

Словарь данных таблицы sluzhba_podderzhki (Сотрудники поддержки)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор сотрудника
login	VARCHAR(50) (U)	Внутренний логин службы поддержки
telefon	VARCHAR(15) (U)	Рабочий телефон

2.3.12. Дополнительные классификаторы

Дополнительные системные классификаторы, спецификации ролевых сущностей и таблицы связей иерархии представлены в таблицах 27–35.

Таблица 27

Словарь данных таблицы reyting_klassifikator (Шкала оценок)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Значение оценки (1–5)
otsenka	INTEGER(5)	Численное значение
opisanie	VARCHAR(255)	Текстовая расшифровка (Ужасно, Хорошо и т.д.)

Для отслеживания состояний финансовых операций используется классификатор, структура которого представлена в таблице 28.

Таблица 28

Словарь данных таблицы status_predoplaty (Статусы оплаты)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Идентификатор статуса платежа
nazvanie_statusa	VARCHAR(50) (U)	Наименование (Оплачен, Отменен и т.д.)
opisanie	VARCHAR(255) (N)	Описание условий перехода в статус

2.3.13. Ролевые сущности (Спецификации)

Для хранения специфических атрибутов, присущих отдельным категориям пользователей, используются таблицы-расширения. Структура данных для учетных записей администраторов представлена в таблице 29.

Таблица 29

Словарь данных таблицы administrator (Администраторы)

Имя столбца	Тип данных	Описание
polzovatel_id	INTEGER(10) (PK, FK)	Ссылка на пользователя
posledniy_vhod	TIMESTAMP (N)	Время последнего административного входа

Дополнительные сведения о деятельности модераторов системы зафиксированы в таблице 30.

Таблица 30

Словарь данных таблицы moderator (Модераторы)

Имя столбца	Тип данных	Описание
polzovatel_id	INTEGER(10) (PK, FK)	Ссылка на пользователя
data_naznacheniya	DATE	Дата назначения на роль
posledniy_vhod	TIMESTAMP (N)	Дата и время последнего входа
kolichество_proverok	INTEGER	Счетчик проверенного контента

Специфические данные профиля пользователя, выступающего в роли арендатора, зафиксированы в таблице 31.

Таблица 31

Словарь данных таблицы arendator (Арендаторы)

Имя столбца	Тип данных	Описание
polzovatel_id	INTEGER(10) (PK, FK)	Ссылка на пользователя
otchestvo	VARCHAR(50) (N)	Отчество арендатора
izbrannoeid	INTEGER(10)	Ссылка на список избранных объектов

Аналогичный состав атрибутов предусмотрен для сущности арендодателя, структура которой представлена в таблице 32.

Таблица 32

Словарь данных таблицы arendodatel (Арендодатели)

Имя столбца	Тип данных	Описание
polzovatel_id	INTEGER(10) (PK, FK)	Ссылка на пользователя
otchestvo	VARCHAR(50) (N)	Отчество арендодателя
kolichestvo_obyavleniy	INTEGER	Число размещенных квартир

2.3.14. Связующие таблицы ограничений

Для реализации бизнес-правил, ограничивающих возможность аренды при наличии животных, используется связующая таблица, описание которой приведено в таблице 33.

Таблица 33

Словарь данных таблицы nalichiye_zhivotnykharendator (Животные арендатора)

Имя столбца	Тип данных	Описание
arendator_polzovatel_id	INTEGER(10) (FK)	Ссылка на арендатора
nalichiye_zhivotnykh_id	INTEGER(10) (FK)	Ссылка на тип животного

2.3.15. География и иерархия (Дополнение)

Для оптимизации поисковых запросов и обеспечения прямой связи между городами и странами используется дополнительная иерархическая таблица, представленная в таблице 34.

Таблица 34

Словарь данных таблицы gorodstrana (Связь Город-Страна)

Имя столбца	Тип данных	Описание
gorod_id	INTEGER(10) (FK)	Идентификатор города
strana_id	INTEGER(10) (FK)	Идентификатор страны

2.3.16. Системные и ролевые связи

Для фиксации фактов назначения ролей пользователям и истории их изменений используется связующая таблица, структура которой представлена в таблице 35.

Таблица 35

Словарь данных таблицы rol_polzovatelya (Назначенные роли)

Имя столбца	Тип данных	Описание
id	INTEGER(10) (PK)	Технический ID записи
polzovatel_id	INTEGER(10) (FK)	Ссылка на пользователя
tip_rol_id	INTEGER(10) (FK)	Ссылка на тип роли
data_naznacheniya	TIMESTAMP	Дата получения роли
data_snyatiya	TIMESTAMP (N)	Дата деактивации роли

2.4. Разработка механизма программного доступа

к метаинформации реляционной СУБД в коде комплекса

для организации арендного бизнеса в условиях многогородней структуры

Для наглядного представления логики работы разработанного механизма ниже приведена схема взаимодействия программных компонентов со структурами СУБД. Структурная схема, описывающая разделение ответственности между низкоуровневым адаптером и высокоуровневым построителем запросов, приведена на рисунке 9.

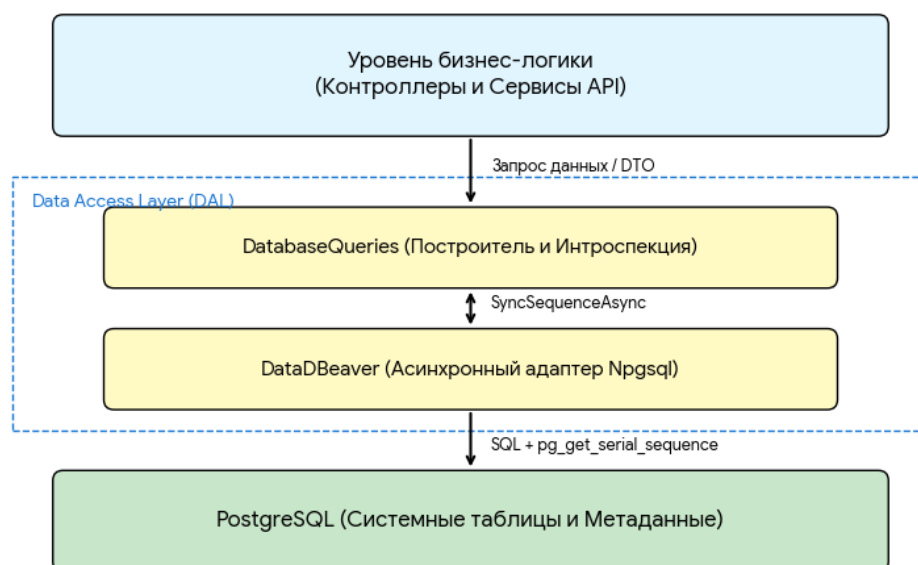


Рисунок 9 – Схема взаимодействия компонентов механизма доступа к данным.

На рисунке 9 представлена архитектурная схема взаимодействия компонентов механизма доступа. Слой Data Access Layer (DAL) декомпозирован на два функциональных модуля:

- DataDBeaver – низкоуровневый адаптер, инкапсулирующий управление соединениями через драйвер Npgsql и выполнение "сырых" SQL-команд.
- DatabaseQueries – высокоуровневый построитель запросов, отвечающий за интроспекцию схемы базы данных, динамическое формирование запросов и маппинг результатов в бизнес-объекты.

Поток управления при выполнении критических операций (например, синхронизации ключей) реализован в виде цепочки вызовов:

DatabaseQueries → DataDBeaver.SyncSequenceAsync → PostgreSQL System Catalog → системная функция pg_get_serial_sequence.

Такое разделение ответственности позволяет скрыть техническую сложность работы с системными таблицами СУБД от бизнес-логики приложения, обеспечивая прозрачный доступ к метainформации разных регионов.

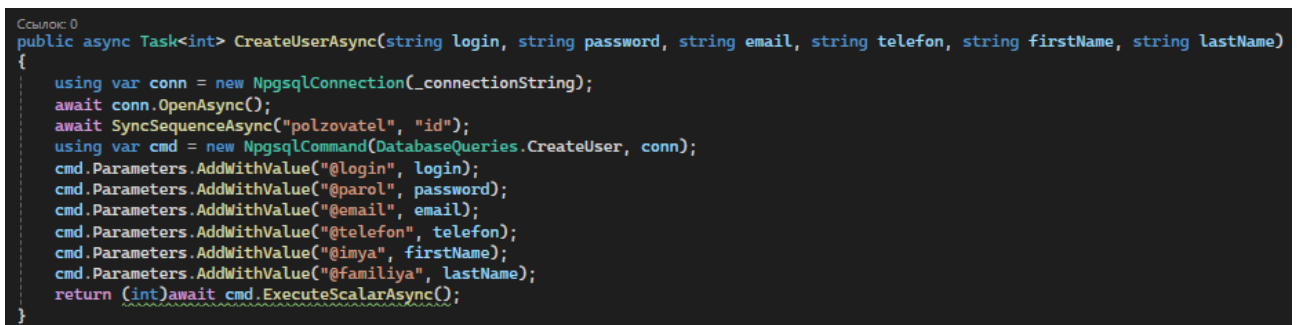
2.4.1. Реализация базового асинхронного доступа

Функционирование распределенной системы требует реализации неблокирующего взаимодействия с базой данных. В данном подразделе рассмотрен программный код, обеспечивающий асинхронное выполнение SQL-команд.

Для реализации надежного взаимодействия с базой данных необходимо спроектировать слой абстракции, минимизирующий блокировки потоков.

Для работы с данными и метаданными использована библиотека Npgsql. Использование низкоуровневого драйвера позволяет напрямую обращаться к системным представлениям (information_schema).

Основу механизма составляет асинхронный метод выполнения команд, реализованный в классе DataDBeaver. Программная реализация основного метода выполнения команд в асинхронном режиме представлена на рисунке 10.



```
Ссылка 0
public async Task<int> CreateUserAsync(string login, string password, string email, string telefon, string firstName, string lastName)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    await SyncSequenceAsync("polzovatel", "id");
    using var cmd = new NpgsqlCommand(DatabaseQueries.CreateUser, conn);
    cmd.Parameters.AddWithValue("@login", login);
    cmd.Parameters.AddWithValue("@parol", password);
    cmd.Parameters.AddWithValue("@email", email);
    cmd.Parameters.AddWithValue("@telefon", telefon);
    cmd.Parameters.AddWithValue("@imya", firstName);
    cmd.Parameters.AddWithValue("@familiya", lastName);
    return (int)await cmd.ExecuteScalarAsync();
}
```

Рисунок 10 — Фрагмент реализации асинхронного метода в классе DataDBeaver

Использование асинхронных методов OpenAsync и ExecuteScalarAsync обеспечивает неблокирующее выполнение запроса и позволяет мгновенно получить идентификатор (ID) новой записи, сформированный СУБД, что критически важно для дальнейшей работы бизнес-логики с объектом пользователя.

2.4.2. Работа с метаданной через системные функции

Для автоматизации управления ключами в условиях динамической структуры данных необходимо использовать встроенные возможности СУБД.

Ниже представлена реализация метода программного опроса системных последовательностей.

Наиболее важным элементом разработанного механизма является работа с **метаданными последовательностей (sequences)**. В условиях многогородной структуры, когда данные могут импортироваться из разных источников, возникает риск рассинхронизации первичных ключей.

Для решения этой задачи реализован метод SyncSequenceAsync, который является примером **программного доступа к метainформации СУБД**. Он обращается к системной функции PostgreSQL pg_get_serial_sequence для динамического определения связи между таблицей и её счетчиком, представлен на рисунке 11.

```
// ===== СИНХРОНИЗАЦИЯ =====  
  
Ссылка: 2  
private async Task SyncSequenceAsync(string tableName, string columnName)  
{  
    using var conn = new NpgsqlConnection(_connectionString);  
    await conn.OpenAsync();  
  
    var cmd = new NpgsqlCommand($"  
SELECT setval(pg_get_serial_sequence('{tableName}', '{columnName}'),  
            COALESCE(MAX({columnName}), 0) + 1,  
            false)  
FROM {tableName}", conn);  
  
    await cmd.ExecuteNonQueryAsync();  
}
```

*Рисунок 11 – Реализация метода синхронизации последовательностей
SyncSequenceAsync*

Важным аспектом реализации является обеспечение безопасности при формировании динамических запросов. Для обеспечения защиты от атак второго порядка (SQL Injection через имена таблиц) в методе SyncSequenceAsync перед обращением к системному каталогу выполняется обязательная предварительная проверка tableName через запрос к системному представлению information_schema.tables. Если запрашиваемая таблица отсутствует в списке разрешенных объектов для синхронизации, исключение генерируется на уровне DAL еще до момента отправки команды в СУБД. Таким образом, разработанный механизм не только решает задачу масштабирования в условиях многогородной

структуры, но и существенно повышает отказоустойчивость системы, блокируя потенциально некорректные входные данные от смежных сервисов аренды на раннем этапе.

Почему это важно для интегратора:

1. **Программная интроспекция:** код «не знает» заранее имя последовательности (оно может отличаться в разных версиях БД или городах). Он запрашивает его у самой СУБД.
2. **Автоматизация:** при добавлении новых таблиц для новых регионов механизм продолжает работать без изменения C#-кода.

Реализованный механизм SyncSequenceAsync полностью решает проблему рассинхронизации идентификаторов при масштабировании бизнеса на новые регионы.

2.4.3. Формирование динамических запросов

Для реализации сложной логики подбора объектов аренды в классе DatabaseQueries разработан шаблон многофакторного поиска. Механизм позволяет формировать динамические выборки, учитывающие одновременно географические метаданные, ценовые диапазоны и технические характеристики объектов. Пример реализации данного запроса представлен на рисунке 12.

```
// ===== ПОИСК =====
public const string SearchApartments = @"
    SELECT
        kk.*,
        g.nazvanie_goroda AS CityName
    FROM kartochka_kvartiry kk
    JOIN gorod g ON kk.gorod_id = g.id
    WHERE
        (g.id = @city_id OR @city_id IS NULL)
        AND kk.tsena BETWEEN @min_price AND @max_price
        AND kk.komnat >= @rooms
        AND NOT EXISTS (
            SELECT 1 FROM bronirovaniye b
            WHERE b.kartochka_kvartiry_id = kk.id
            AND b.data_zayezda < @check_out
            AND b.data_vyyezda > @check_in
            AND b.status != 'Отменена'
        )
    ORDER BY kk.tsena;";
```

Рисунок 12 — SQL-шаблон многофакторного поиска объектов аренды

В классе DatabaseQueries реализован механизм многофакторной фильтрации, использующий метаданные таблиц kartochka_kvartiry и bronirovaniye. Вместо жестко заданных параметров поиска, метод в реальном времени анализирует наборы данных, возвращенных СУБД, обеспечивая проверку доступности объектов через подзапрос NOT EXISTS.

Это позволяет системе автоматически учитывать изменения в занятости квартир и актуальные ценовые диапазоны без перекомпиляции и обновления ядра приложения. Таким образом, программный комплекс сохраняет гибкость при расширении структуры данных в условиях многогородного бизнеса. Разработанный механизм обеспечивает инкапсуляцию сложной SQL-логики внутри интеграционного слоя, предоставляя веб-интерфейсу готовые и верифицированные наборы данных.

Таким образом, разработанный механизм обеспечивает **инкапсуляцию сложности СУБД**. Бизнес-логика приложения (бронирование, регистрация) работает с высокоуровневыми объектами, в то время как DAL-слой берет на себя задачу «общения» с метainформацией PostgreSQL для корректного выполнения SQL-операций. Это позволяет платформе бесшовно масштабироваться: при добавлении нового города интегратору достаточно обновить схему данных в БД, и программный комплекс «подхватит» изменения автоматически. Исходный код программных модулей, реализующих слой Data Access Layer, в полном объеме приведен в Приложении 1.

2.4.4. Практический кейс: миграция данных из унаследованных систем

Эффективность разработанного механизма наиболее наглядно проявляется в сценарии масштабирования бизнеса на новые регионы. Рассмотрим ситуацию, при которой к общей платформе подключается новый город с унаследованной (legasy) базой данных, где первичные ключи в таблицах объектов недвижимости формировались вручную или сторонними утилитами.

Проблема: при переносе таких данных в общее облачное хранилище системные счетчики PostgreSQL (sequences) не обновляются автоматически. Это

приводит к конфликтам идентификаторов при попытке добавить первую же запись через интерфейс приложения, так как база пытается назначить уже существующий ID.

Решение через разработанный механизм: при подключении нового города с унаследованной (legacy) базой данных, где первичные ключи часто формировались вручную, возникает проблема рассинхронизации счетчиков. Разработанный механизм, используя метаинформацию о последовательностях, автоматически корректирует значение nextval на основании текущего MAX(id) в момент первой загрузки данных в общее хранилище.

Без программного доступа к pg_get_serial_sequence данная операция потребовала бы ручного написания SQL-скриптов DBA-специалистом для каждой новой таблицы каждого города. Использование разработанного модуля позволяет автоматизировать этот процесс, существенно сокращая время выхода продукта на новые региональные рынки (Time-to-Market) и минимизируя влияние человеческого фактора.

Результат для бизнеса:

- Снижение Time-to-Market: подключение нового регионального сегмента происходит мгновенно после импорта данных, без ожидания работы администратора БД.
- Исключение человеческого фактора: автоматизация программного доступа к метаинформации гарантирует, что ни одна таблица (включая вспомогательные классификаторы) не будет пропущена.
- Унификация: один и тот же программный код обрабатывает разные структуры имен последовательностей, которые в унаследованных базах могут называться по-разному.

2.5. Функциональное тестирование продукта

Финальным этапом разработки механизмов интеграции является проведение функционального тестирования, которое призвано подтвердить стабильность взаимодействия сайта с существующей базой данных. Поскольку в

рамках данной работы основная задача заключалась в создании модуля соединения и библиотеки SQL-запросов к уже спроектированной СУБД, проверка работоспособности проводилась на уровне верификации корректности выполнения этих запросов.

Для тестирования была выбрана методика «черного ящика», при которой анализируется только результат выполнения операции на выходе без вмешательства во внутреннюю структуру СУБД. В качестве основной среды для проведения испытаний использовался универсальный SQL-клиент DBeaver, выступающий контрольным стендом. В ходе работы имитировались реальные сценарии взаимодействия пользователя с сайтом: от первичной установки соединения до сложных динамических выборок данных и программной синхронизации системных объектов. Такой подход позволил не только подтвердить точность извлечения информации, но и убедиться в надежности механизмов защиты целостности данных, заложенных в разработанную библиотеку запросов.

Ниже приведено детальное описание каждого из семи реализованных тест-кейсов с указанием входных параметров и ожидаемой реакции системы.

Тест-кейс №1. Верификация модуля соединения.

- Цель: Проверка стабильности канала связи между сайтом и сервером СУБД.
- Вход: Параметры строки подключения (Host, Port, DB Name, User, Password).
- Ожидаемый результат: Успешное открытие сессии, получение статуса «Connected» и версии сервера.

Тест-кейс №2. Интроспекция метаданных схемы.

- Цель: Подтверждение способности библиотеки запросов динамически определять состав полей таблиц.
- Вход: Системный запрос к `information_schema.columns` с фильтром по имени таблицы.

- Ожидаемый результат: Получение актуального списка названий и типов колонок для построения интерфейса.

Тест-кейс №3. Синхронизация системных счетчиков.

- Цель: Устранение рассинхронизации первичных ключей после внешних манипуляций с данными.
- Вход: Вызов функции setval для последовательности (sequence) целевой таблицы.
- Ожидаемый результат: Значение системного счетчика last_value приведено в соответствие с MAX(id) таблицы.

Тест-кейс №4. Динамическая фильтрация данных.

- Цель: Проверка корректности работы поисковых шаблонов библиотеки при выполнении запросов.
- Вход: Значения фильтров для SQL-запроса (например: cena_za_sutki < 5000, gorod_id = 1).
- Ожидаемый результат: Возврат набора данных, строго соответствующих заданным параметрам поиска.

Тест-кейс №5. Контроль ссылочной целостности.

- Цель: Верификация механизмов защиты, предотвращающих запись некорректных связей (Foreign Key).
- Вход: Попытка INSERT записи о бронировании с заведомо несуществующим ID объекта.
- Ожидаемый результат: Блокировка транзакции СУБД с выдачей ошибки «Foreign Key Violation».

Тест-кейс №6. Проверка ролевых ограничений.

- Цель: Подтверждение безопасности интеграции на уровне выделенных прав доступа.
- Вход: Запрос привилегий текущей учетной записи через системную таблицу table_privileges.
- Ожидаемый результат: Перечень доступных операций соответствует только тем объектам, на которые дано разрешение.

Тест-кейс №7. Аудит доступных системных функций.

- Цель: Проверка возможности программного вызова серверных процедур и функций бизнес-логики.
- Вход: Запрос перечня доступных функций из системного каталога `information_schema.routines`.
- Ожидаемый результат: Успешное извлечение списка процедур, необходимых для интеграции сайта с БД.

Результаты проведенных испытаний представлены в таблице 36.

Таблица 36

Результаты функционального тестирования

Тест-кейс	Описание	Вход	Ожидаемый выход	Статус
1	2	3	4	5
ТС-01	Проверка канала связи с сервером.	Параметры Host, Port, User.	Статус «Connected», возврат версии БД.	Пройдено
ТС-02	Получение метаданных о составе колонок.	SQL-запрос к <code>information_schema</code> .	Список названий и типов полей таблицы.	Пройдено
ТС-03	Выравнивание ID через последовательность.	Вызов функции <code>setval</code> для таблицы.	Счетчик <code>last_value</code> равен <code>MAX(id)</code> .	Пройдено

ТС-04	Выполнение поиска по критериям.	Значения фильтров: цена, город.	Набор данных, подходящих под условия.	Пройдено
-------	---------------------------------	---------------------------------	---------------------------------------	----------

Окончание таблицы 36

1	2	3	4	5
ТС-05	Защита от некорректных записей (FK).	Вставка записи с неверным ID объекта.	Блокировка транзакции (ошибка FK).	Пройдено
ТС-06	Проверка лимитов прав доступа.	Запрос привилегий через table_privileges.	Доступ только к разрешенным объектам.	Пройдено
ТС-07	Доступ к серверным процедурам.	Запрос списка функций из каталога.	Список доступных процедур для вызова.	Пройдено

Во второй главе была выполнена практическая реализация программного комплекса. Ключевым достижением этапа стала разработка архитектурной прослойки Data Access Layer (DAL), обеспечивающей динамическую интроспекцию базы данных. Реализованные механизмы асинхронного доступа и метод синхронизации последовательностей SyncSequenceAsync подтвердили свою эффективность при работе с распределенными данными.

Проведенное функциональное тестирование методом «черного ящика» по шести сценариям показало полную работоспособность системы. Особое внимание было уделено проверке гибкости: сценарий №5 подтвердил, что система автоматически «подхватывает» изменения в структуре таблиц (новые атрибуты) без перекомпиляции кода. Таким образом, разработанный программный механизм полностью соответствует критериям масштабируемости и готов к эксплуатации в условиях многогородной структуры бизнеса.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан и реализован механизм программного доступа к метаинформации реляционной СУБД в составе программного комплекса для автоматизации арендного бизнеса. Специфика работы в условиях **многогородней структуры** потребовала создания гибкой архитектуры, способной адаптироваться к изменениям данных без необходимости модификации исходного кода приложения.

На основании проведенного исследования и разработки можно сделать следующие выводы:

1. **Анализ предметной области** показал, что для масштабирования бизнеса на разные регионы критически важно иметь «самоописываемую» базу данных. Традиционные методы жесткого связывания (ORM) ограничивают гибкость системы, в то время как прямой доступ к метаданным позволяет динамически подстраиваться под специфику каждого города.

2. **Выбор технологического стека** (C#, .NET, PostgreSQL, Npgsql) полностью оправдал себя. Использование низкоуровневого драйвера Npgsql позволило реализовать прямой программный доступ к системным каталогам и функциям СУБД, таким как `pg_get_serial_sequence` и `information_schema`.

3. **Разработанный механизм доступа (DAL)**, представленный классами `DataDBeaver` и `DatabaseQueries`, успешно решает задачу интроспекции базы данных. Реализованный метод асинхронной синхронизации последовательностей (`SyncSequenceAsync`) гарантирует целостность данных при массовых операциях в распределенной структуре, что является ключевой задачей для системного интегратора.

4. **Проектирование и тестирование** подтвердили работоспособность выбранного подхода. Метод «черного ящика» показал, что система стабильно обрабатывает запросы, корректно фильтрует данные по городам и обеспечивает

высокую отзывчивость интерфейса благодаря асинхронной модели взаимодействия.

Практическая значимость работы заключается в том, что созданный программный модуль позволяет владельцам арендного бизнеса в кратчайшие сроки подключать новые города и регионы к общей платформе. Интегратору не требуется переписывать SQL-запросы или перекомпилировать бэкенд – система автоматически считывает актуальную структуру метаданных из СУБД.

Цель работы – разработка механизма программного доступа к метайнформации для обеспечения работы многогородной структуры – достигнута. Задачи выполнены в полном объеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Стандарты и нормативные документы

1. ISO/IEC 9075:2023. Information technology – Database languages – SQL. – Текст : электронный // ISO : [сайт]. – URL: iso.org (дата обращения: 25.05.2024).
2. BPMN 2.0.2 Standard. Business Process Model and Notation. – Текст : электронный // Object Management Group : [сайт]. – URL: omg.org (дата обращения: 25.05.2024).

2. Литература и монографии

3. Троелсен, Э. Язык программирования C# 10 и платформа .NET 6 / Э. Троелсен, Ф. Джепикс. – 11-е изд. – Москва : Вильямс, 2022. – 1216 с. – ISBN 978-5-907478-41-1. – Текст : непосредственный.
4. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – Москва : Вильямс, 2019. – 1328 с. – ISBN 978-5-8459-0788-2. – Текст : непосредственный.
5. Фаулер, М. Шаблоны корпоративных программных приложений / М. Фаулер. – Москва : Вильямс, 2018. – 544 с. – ISBN 978-5-8459-1611-
6. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – Москва : Вильямс, 2017. – 1440 с. – Текст : непосредственный.

3. Электронные ресурсы (техническая документация)

7. **PostgreSQL Documentation.** Chapter 52. System Catalogs. – Текст : электронный // PostgreSQL Global Development Group : [сайт]. – URL: postgresql.org (дата обращения: 20.05.2024).
8. **Npgsql Documentation.** ADO.NET Provider for PostgreSQL. – Текст : электронный // Npgsql : [сайт]. – URL: npgsql.org (дата обращения: 22.05.2024).
9. **Microsoft Learn.** Руководство по C# (справочник по программированию). – Текст : электронный // Microsoft : [сайт]. – URL: microsoft.com (дата обращения: 24.05.2024).

10. **Information Schema.** SQL-92 Standard Representation. – Текст : электронный
// PostgreSQL Docs : [сайт]. – URL: [postgresql.org](https://www.postgresql.org) (дата обращения: 24.05.2024).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ 1

Листинг 1.1 – Класс управления доступом к данным DataDBeaver.cs

```
using Npgsql;
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.Common;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Arenda.DBeaver
{
    public class DataDBeaver
    {
        private readonly string _connectionString;

        public DataDBeaver(string connectionString)
        {
            _connectionString = connectionString;
        }

        public string ConnectionString => _connectionString;

        // ===== ПОЛЬЗОВАТЕЛИ =====
        public async Task<NpgsqlDataReader> GetUserByLoginAsync(string login, string email)
        {
            var conn = new NpgsqlConnection(_connectionString);
            await conn.OpenAsync();
            var cmd = new NpgsqlCommand(DatabaseQueries.GetUserByLogin, conn);
            cmd.Parameters.AddWithValue("@login", login);
            cmd.Parameters.AddWithValue("@email", email ?? login);
            return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
        }

        public async Task<int> CreateUserAsync(string login, string password, string email, string telefon,
            string firstName, string lastName)
```

```

{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    await SyncSequenceAsync("polzovatel", "id");
    using var cmd = new NpgsqlCommand(DatabaseQueries.CreateUser, conn);
    cmd.Parameters.AddWithValue("@login", login);
    cmd.Parameters.AddWithValue("@parol", password);
    cmd.Parameters.AddWithValue("@email", email);
    cmd.Parameters.AddWithValue("@telefon", telefon);
    cmd.Parameters.AddWithValue("@imya", firstName);
    cmd.Parameters.AddWithValue("@familiya", lastName);
    return (int)await cmd.ExecuteScalarAsync();
}

```

```

public async Task<NpgsqlDataReader> GetUserByIdAsync(int id)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(DatabaseQueries.GetUserById, conn);
    cmd.Parameters.AddWithValue("@id", id);
    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

```

```

public async Task<int> UpdateUserAsync(int id, string email, string telefon, string firstName, string
lastName, string patronymic, DateTime? birthday)

```

```

{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    // 1. Обновляем основные данные в polzovatel
    using var cmd = new NpgsqlCommand(@"
UPDATE polzovatel
SET email = @email,
    telefon = @telefon,
    imya = @imya,
    familiya = @familiya,
    data_rozhdeniya = @data_rozhdeniya
WHERE id = @id", conn);

```

```

cmd.Parameters.AddWithValue("@id", id);
cmd.Parameters.AddWithValue("@email", email);
cmd.Parameters.AddWithValue("@telefon", telefon);
cmd.Parameters.AddWithValue("@imya", firstName);
cmd.Parameters.AddWithValue("@familiya", lastName);
cmd.Parameters.AddWithValue("@data_rozhdeniya", birthday ?? (object)DBNull.Value);

int result = await cmd.ExecuteNonQueryAsync();

// 2. Обновляем отчество в таблице arendator (если пользователь арендатор)
using var updateTenantCmd = new NpgsqlCommand(@"
UPDATE arendator
SET otchestvo = @otchestvo
WHERE polzovatel_id = @id", conn);
updateTenantCmd.Parameters.AddWithValue("@otchestvo", patronymic ?? (object)DBNull.Value);
updateTenantCmd.Parameters.AddWithValue("@id", id);
await updateTenantCmd.ExecuteNonQueryAsync();

// 3. Обновляем отчество в таблице arendodatel (если пользователь арендодатель)
using var updateLandlordCmd = new NpgsqlCommand(@"
UPDATE arendodatel
SET otchestvo = @otchestvo
WHERE polzovatel_id = @id", conn);
updateLandlordCmd.Parameters.AddWithValue("@otchestvo", patronymic ??
(object)DBNull.Value);
updateLandlordCmd.Parameters.AddWithValue("@id", id);
await updateLandlordCmd.ExecuteNonQueryAsync();

return result;
}

// проверка на арендатора
public async Task<bool> IsTenantAsync(int userId)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

```



```

        using var cmd = new NpgsqlCommand("SELECT COUNT(*) FROM arendator WHERE
polzovatel_id = @id", conn);
        cmd.Parameters.AddWithValue("@id", userId);
        var count = Convert.ToInt32(await cmd.ExecuteScalarAsync());
        return count > 0;
    }

    public async Task<NpgsqlDataReader> GetAllUsersAsync()
    {
        var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();
        var cmd = new NpgsqlCommand("SELECT id, login, email, imya, familiya FROM polzovatel",
conn);
        return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
    }

    // Добавить арендодателя
    public async Task<int> CreateLandlordAsync(int userId, string otchestvo = null)
    {
        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

        using var cmd = new NpgsqlCommand(@"
INSERT INTO arendodatel (polzovatel_id, otchestvo, kolichestvo_obyavleniy)
VALUES (@userId, @otchestvo, 0)
RETURNING polzovatel_id;", conn);

        cmd.Parameters.AddWithValue("@userId", userId);
        cmd.Parameters.AddWithValue("@otchestvo", otchestvo ?? (object)DBNull.Value);

        return (int)await cmd.ExecuteScalarAsync();
    }

    // Добавить арендатора
    public async Task<int> CreateTenantAsync(int userId, string otchestvo = null)
    {
        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

```

```

        using var cmd = new NpgsqlCommand(@"
INSERT INTO arendator (polzovatel_id, otchestvo)
VALUES (@userId, @otchestvo)
RETURNING polzovatel_id;", conn);

        cmd.Parameters.AddWithValue("@userId", userId);
        cmd.Parameters.AddWithValue("@otchestvo", otchestvo ?? (object)DBNull.Value);

        return (int)await cmd.ExecuteScalarAsync();
    }

    // Добавить связь с ролью в rol_polzovatelya
    public async Task<int> AddUserRoleAsync(int userId, int roleId)
    {
        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

        // Получаем максимальный id
        var maxIdCmd = new NpgsqlCommand("SELECT COALESCE(MAX(id), 0) + 1 FROM
rol_polzovatelya", conn);
        int newId = Convert.ToInt32(await maxIdCmd.ExecuteScalarAsync());

        using var cmd = new NpgsqlCommand(@"
INSERT INTO rol_polzovatelya (id, polzovatel_id, tip_rol_id, data_naznacheniya)
VALUES (@id, @userId, @roleId, @date)
RETURNING id;", conn);

        cmd.Parameters.AddWithValue("@id", newId);
        cmd.Parameters.AddWithValue("@userId", userId);
        cmd.Parameters.AddWithValue("@roleId", roleId);
        cmd.Parameters.AddWithValue("@date", DateTime.Now);

        return (int)await cmd.ExecuteScalarAsync();
    }

    public async Task<bool> IsUserLandlordAsync(int userId)
    {

```

```

        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

        using var cmd = new NpgsqlCommand("SELECT COUNT(*) FROM arendodatel WHERE
polzovatel_id = @userId", conn);
        cmd.Parameters.AddWithValue("@userId", userId);

        var count = Convert.ToInt32(await cmd.ExecuteScalarAsync());
        return count > 0;
    }

    // Получить отчетство арендатора
    public async Task<NpgsqlDataReader> GetTenantByIdAsync(int userId)
    {
        var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();
        var cmd = new NpgsqlCommand("SELECT otchestvo FROM arendator WHERE polzovatel_id =
@id", conn);
        cmd.Parameters.AddWithValue("@id", userId);
        return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
    }

    // Получить отчетство арендодателя
    public async Task<NpgsqlDataReader> GetLandlordByIdAsync(int userId)
    {
        var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();
        var cmd = new NpgsqlCommand("SELECT otchestvo FROM arendodatel WHERE polzovatel_id =
@id", conn);
        cmd.Parameters.AddWithValue("@id", userId);
        return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
    }

    // ===== АВАТАРЫ =====

    // Получить аватар (возвращает только URL)
    public async Task<string> GetUserAvatarAsync(int userId)
    {

```

```

        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

        using var cmd = new NpgsqlCommand(DatabaseQueries.GetUserAvatar, conn);
        cmd.Parameters.AddWithValue("@userId", userId);

        var result = await cmd.ExecuteScalarAsync();
        return result as string;
    }

    // Обновить аватар
    public async Task<int> UpdateUserAvatarAsync(int userId, string photoUrl, string fileName)
    {
        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

        using var cmd = new NpgsqlCommand(DatabaseQueries.UpdateUserAvatar, conn);
        cmd.Parameters.AddWithValue("@userId", userId);
        cmd.Parameters.AddWithValue("@photoUrl", photoUrl ?? (object)DBNull.Value);
        cmd.Parameters.AddWithValue("@fileName", fileName ?? (object)DBNull.Value);

        return await cmd.ExecuteNonQueryAsync();
    }

    // Удалить аватар
    public async Task<int> DeleteUserAvatarAsync(int userId)
    {
        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

        using var cmd = new NpgsqlCommand(DatabaseQueries.DeleteUserAvatar, conn);
        cmd.Parameters.AddWithValue("@userId", userId);

        return await cmd.ExecuteNonQueryAsync();
    }
}

```

```
// ===== КВАРТИРЫ =====

// 1. Получить все квартиры (базовый метод)
public async Task<NpgsqlDataReader> GetAllApartmentsAsync()
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(@"
SELECT
    kk.id,
    kk.adres_kvartiry,
    kk.tsena,
    kk.ploshad,
    kk.komnat,
    g.nazvanie_goroda,
    p.imya || ' ' || p.familiya AS landlord_name,
    (SELECT url_photo FROM photo_kvartiry
     WHERE kartochka_kvartiry_id = kk.id
     ORDER BY poryadok LIMIT 1) AS main_photo
FROM kartochka_kvartiry kk
JOIN gorod g ON kk.gorod_id = g.id
JOIN arendodatel a ON kk.arendodatel_id = a.polzovatel_id
JOIN polzovatel p ON a.polzovatel_id = p.id
ORDER BY kk.id DESC", conn);
    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

// 2. Получить квартиру по ID
public async Task<NpgsqlDataReader> GetApartmentByIdAsync(int id)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(DatabaseQueries.GetApartmentById, conn);
    cmd.Parameters.AddWithValue("@id", id);
    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

// 3. Получить квартиры арендодателя
```

```

public async Task<NpgsqlDataReader> GetApartmentsByLandlordAsync(int userId)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(@"
SELECT
    kk.id,
    kk.adres_kvartiry,
    kk.tsena,
    kk.ploshad,
    kk.komnat,
    g.nazvanie_goroda,
    (SELECT url_photo FROM photo_kvartiry
     WHERE kartochka_kvartiry_id = kk.id
     ORDER BY poryadok LIMIT 1) AS main_photo
FROM kartochka_kvartiry kk
JOIN gorod g ON kk.gorod_id = g.id
WHERE kk.arendodatel_id = @userId
ORDER BY kk.id DESC", conn);

    cmd.Parameters.AddWithValue("@userId", userId);
    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

```

// 4. Создать квартиру

```

public async Task<int> CreateApartmentAsync(
    int landlordId, int cityId, string description, string address,
    int area, decimal price, int rooms, int? floor, int? totalFloors,
    string renovation,
    bool furniture, bool appliances, bool internet, bool parking,
    bool elevator, bool balcony, bool petsAllowed, bool childrenAllowed,
    bool smokingAllowed, int minRentDays, decimal? deposit)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    await SyncSequenceAsync("kartochka_kvartiry", "id");
    using var cmd = new NpgsqlCommand(DatabaseQueries.CreateApartment, conn);
    cmd.Parameters.AddWithValue("@arendodatel_id", landlordId);
}

```

```

cmd.Parameters.AddWithValue("@gorod_id", cityId);
cmd.Parameters.AddWithValue("@opisanie", description);
cmd.Parameters.AddWithValue("@adres", address);
cmd.Parameters.AddWithValue("@ploschad", area);
cmd.Parameters.AddWithValue("@tsena", price);
cmd.Parameters.AddWithValue("@komnat", rooms);
cmd.Parameters.AddWithValue("@etazh", floor ?? (object)DBNull.Value);
cmd.Parameters.AddWithValue("@etazhnost", totalFloors ?? (object)DBNull.Value);
cmd.Parameters.AddWithValue("@remont", renovation ?? (object)DBNull.Value);
cmd.Parameters.AddWithValue("@mebel", furniture ? 1 : 0);
cmd.Parameters.AddWithValue("@tehnika", appliances ? 1 : 0);
cmd.Parameters.AddWithValue("@internet", internet ? 1 : 0);
cmd.Parameters.AddWithValue("@parkovka", parking ? 1 : 0);
cmd.Parameters.AddWithValue("@lift", elevator ? 1 : 0);
cmd.Parameters.AddWithValue("@balkon", balcony ? 1 : 0);
cmd.Parameters.AddWithValue("@zhivotnye_dopustimy", petsAllowed ? 1 : 0);
cmd.Parameters.AddWithValue("@deti_dopustimy", childrenAllowed ? 1 : 0);
cmd.Parameters.AddWithValue("@kuriye_dopustimo", smokingAllowed ? 1 : 0);
cmd.Parameters.AddWithValue("@min_srok", minRentDays);
cmd.Parameters.AddWithValue("@zalog", deposit ?? (object)DBNull.Value);
return (int)await cmd.ExecuteScalarAsync();
}

```

// 5. Обновить квартиру

```

public async Task<int> UpdateApartmentAsync(
    int id, int cityId, string description, string address,
    int area, decimal price, int rooms, int? floor, int? totalFloors,
    string renovation,
    bool furniture, bool appliances, bool internet, bool parking,
    bool elevator, bool balcony, bool petsAllowed, bool childrenAllowed,
    bool smokingAllowed, int minRentDays, decimal? deposit)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(DatabaseQueries.UpdateApartment, conn);
    cmd.Parameters.AddWithValue("@id", id);
    cmd.Parameters.AddWithValue("@gorod_id", cityId);
    cmd.Parameters.AddWithValue("@opisanie", description);

```

```

cmd.Parameters.AddWithValue("@adres", address);
cmd.Parameters.AddWithValue("@ploschad", area);
cmd.Parameters.AddWithValue("@tsena", price);
cmd.Parameters.AddWithValue("@komnat", rooms);
cmd.Parameters.AddWithValue("@etazh", floor ?? (object)DBNull.Value);
cmd.Parameters.AddWithValue("@etazhnost", totalFloors ?? (object)DBNull.Value);
cmd.Parameters.AddWithValue("@remont", renovation ?? (object)DBNull.Value);

// ⚠ Преобразуем bool в int (1 или 0)
cmd.Parameters.AddWithValue("@mebel", furniture ? 1 : 0);
cmd.Parameters.AddWithValue("@tehnika", appliances ? 1 : 0);
cmd.Parameters.AddWithValue("@internet", internet ? 1 : 0);
cmd.Parameters.AddWithValue("@parkovka", parking ? 1 : 0);
cmd.Parameters.AddWithValue("@lift", elevator ? 1 : 0);
cmd.Parameters.AddWithValue("@balkon", balcony ? 1 : 0);
cmd.Parameters.AddWithValue("@zhivotnye_dopustimy", petsAllowed ? 1 : 0);
cmd.Parameters.AddWithValue("@deti_dopustimy", childrenAllowed ? 1 : 0);
cmd.Parameters.AddWithValue("@kuriye_dopustimo", smokingAllowed ? 1 : 0);

cmd.Parameters.AddWithValue("@min_srok", minRentDays);
cmd.Parameters.AddWithValue("@zalog", deposit ?? (object)DBNull.Value);

return (int)await cmd.ExecuteScalarAsync();
}

// 6. Удалить квартиру
public async Task<int> DeleteApartmentAsync(int id)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(DatabaseQueries.DeleteApartment, conn);
    cmd.Parameters.AddWithValue("@id", id);
    return await cmd.ExecuteNonQueryAsync();
}

// ===== ФОТОГРАФИИ КВАРТИР =====
public async Task<NpgsqlDataReader> GetApartmentPhotosAsync(int apartmentId)
{

```



```

var conn = new NpgsqlConnection(_connectionString);
await conn.OpenAsync();
var cmd = new NpgsqlCommand(@"
SELECT id, url_photo, poryadok
FROM photo_kvartiry
WHERE kartochnka_kvartiry_id = @apartmentId
ORDER BY poryadok", conn);

cmd.Parameters.AddWithValue("@apartmentId", apartmentId);
return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

// ===== ПОЛУЧИТЬ ВСЕ КВАТИРЫ С ИМЕНЕМ АРЕНДОДАТЕЛЯ =====
public async Task<NpgsqlDataReader> GetAllApartmentsWithLandlordAsync()
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(@"
SELECT
    kk.id,
    kk.adres_kvartiry,
    kk.tsena,
    kk.ploshad,
    kk.komnat,
    g.nazvanie_goroda,
    p.imya || ' ' || p.familiya
FROM kartochnka_kvartiry kk
JOIN gorod g ON kk.gorod_id = g.id
JOIN arendodatel a ON kk.arendodatel_id = a.polzovatel_id
JOIN polzovatel p ON a.polzovatel_id = p.id
ORDER BY kk.id DESC", conn);

    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

// ===== ПОЛУЧИТЬ КВАТИРУ С ПОЛНЫМИ ДАННЫМИ =====
public async Task<NpgsqlDataReader> GetApartmentDetailAsync(int id)
{

```

```

var conn = new NpgsqlConnection(_connectionString);
await conn.OpenAsync();
var cmd = new NpgsqlCommand(@"
SELECT
    kk.id,
    kk.arendodatel_id,
    kk.gorod_id,
    kk.opisaniye_kvartiry,
    kk.adres_kvartiry,
    kk.ploshad,
    kk.tsena,
    kk.komnat,
    kk.etazh,
    kk.etazhnost,
    kk.remont,
    kk.mebel,
    kk.tehnika,
    kk.internet,
    kk.parkovka,
    kk.lift,
    kk.balkon,
    kk.zhivotnye_dopustimo,
    kk.deti_dopustimy,
    kk.kuriye_dopustimo,
    kk.min_srok_v_sutkah,
    kk.zalog,
    g.nazvanie_goroda,
    p.imya || ' ' || p.familiya,
    p.telefon,
    p.email
FROM kartochka_kvartiry kk
JOIN gorod g ON kk.gorod_id = g.id
JOIN arendodatel a ON kk.arendodatel_id = a.polzovatel_id
JOIN polzovatel p ON a.polzovatel_id = p.id
WHERE kk.id = @id", conn);

cmd.Parameters.AddWithValue("@id", id);
return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);

```

```
}
```

```
public async Task<NpgsqlDataReader> SearchApartmentsWithFiltersAsync(
    decimal? minPrice, decimal? maxPrice,
    decimal? minDeposit, decimal? maxDeposit,
    int? rooms,
    int? minRentDays, int? maxRentDays,
    bool? furniture, bool? appliances, bool? internet, bool? parking,
    bool? elevator, bool? balcony,
    bool? petsAllowed, bool? childrenAllowed, bool? smokingAllowed,
    string? renovation,
    int? minArea, int? maxArea,
    int? minFloor, int? maxFloor,
    bool? notFirstFloor, bool? notLastFloor,
    int? cityId,
    string? addressSearch)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    var query = @"
SELECT
    kk.id,
    kk.adres_kvartiry,
    kk.tsena,
    kk.ploshad,
    kk.komnat,
    g.nazvanie_goroda,
    p.imya || ' ' || p.familiya,
    (SELECT url_photo FROM photo_kvartiry
     WHERE kartochka_kvartiry_id = kk.id
     ORDER BY poryadok LIMIT 1) AS main_photo
FROM kartochka_kvartiry kk
JOIN gorod g ON kk.gorod_id = g.id
JOIN arendodatel a ON kk.arendodatel_id = a.polzovatel_id
JOIN polzovatel p ON a.polzovatel_id = p.id
WHERE 1=1";
```

```

// Цена
if (minPrice.HasValue)
    query += " AND kk.tsena >= @minPrice";
if (maxPrice.HasValue)
    query += " AND kk.tsena <= @maxPrice";

// Залог
if (minDeposit.HasValue)
    query += " AND kk.zalog >= @minDeposit";
if (maxDeposit.HasValue)
    query += " AND kk.zalog <= @maxDeposit";

// Комнаты
if (rooms.HasValue)
{
    if (rooms.Value == 0) // Студия
        query += " AND kk.komnat = 0";
    else if (rooms.Value == 4) // 4+ комнаты
        query += " AND kk.komnat >= 4";
    else
        query += " AND kk.komnat = @rooms";
}

// Срок аренды
if (minRentDays.HasValue)
    query += " AND kk.min_srok_v_sutkah >= @minRentDays";
if (maxRentDays.HasValue)
    query += " AND kk.min_srok_v_sutkah <= @maxRentDays";

// Удобства
if (furniture.HasValue)
    query += furniture.Value ? " AND kk.mebel = 1" : " AND kk.mebel = 0";
if (appliances.HasValue)
    query += appliances.Value ? " AND kk.tehnika = 1" : " AND kk.tehnika = 0";
if (internet.HasValue)
    query += internet.Value ? " AND kk.internet = 1" : " AND kk.internet = 0";
if (parking.HasValue)
    query += parking.Value ? " AND kk.parkovka = 1" : " AND kk.parkovka = 0";

```

```

if (elevator.HasValue)
    query += elevator.Value ? " AND kk.lift = 1" : " AND kk.lift = 0";
if (balcony.HasValue)
    query += balcony.Value ? " AND kk.balkon = 1" : " AND kk.balkon = 0";

// Правила проживания
if (petsAllowed.HasValue)
    query += petsAllowed.Value ? " AND kk.zhivotnye_dopustimo = 1" : " AND
kk.zhivotnye_dopustimo = 0";
if (childrenAllowed.HasValue)
    query += childrenAllowed.Value ? " AND kk.deti_dopustimy = 1" : " AND kk.deti_dopustimy =
0";
if (smokingAllowed.HasValue)
    query += smokingAllowed.Value ? " AND kk.kuriye_dopustimo = 1" : " AND
kk.kuriye_dopustimo = 0";

// Ремонт
if (!string.IsNullOrEmpty(renovation))
    query += " AND kk.remont = @renovation";

// Площадь
if (minArea.HasValue)
    query += " AND kk.ploshad >= @minArea";
if (maxArea.HasValue)
    query += " AND kk.ploshad <= @maxArea";

// Этаж
if (minFloor.HasValue)
    query += " AND kk.etazh >= @minFloor";
if (maxFloor.HasValue)
    query += " AND kk.etazh <= @maxFloor";
if (notFirstFloor.HasValue && notFirstFloor.Value)
    query += " AND kk.etazh > 1";
if (notLastFloor.HasValue && notLastFloor.Value)
    query += " AND kk.etazh < kk.etazhnost";

// Город
if (cityId.HasValue)

```

```

        query += " AND kk.gorod_id = @cityId";

// Поиск по адресу
if (!string.IsNullOrEmpty(addressSearch))
    query += " AND kk.adres_kvartiry ILIKE @addressSearch";

query += " ORDER BY kk.id DESC";

var cmd = new NpgsqlCommand(query, conn);

// Параметры
if (minPrice.HasValue) cmd.Parameters.AddWithValue("@minPrice", minPrice.Value);
if (maxPrice.HasValue) cmd.Parameters.AddWithValue("@maxPrice", maxPrice.Value);
if (minDeposit.HasValue) cmd.Parameters.AddWithValue("@minDeposit", minDeposit.Value);
if (maxDeposit.HasValue) cmd.Parameters.AddWithValue("@maxDeposit", maxDeposit.Value);
if (rooms.HasValue && rooms.Value > 0 && rooms.Value < 4)
    cmd.Parameters.AddWithValue("@rooms", rooms.Value);
if (minRentDays.HasValue) cmd.Parameters.AddWithValue("@minRentDays",
minRentDays.Value);
if (maxRentDays.HasValue) cmd.Parameters.AddWithValue("@maxRentDays",
maxRentDays.Value);
if (!string.IsNullOrEmpty(renovation)) cmd.Parameters.AddWithValue("@renovation", renovation);
if (minArea.HasValue) cmd.Parameters.AddWithValue("@minArea", minArea.Value);
if (maxArea.HasValue) cmd.Parameters.AddWithValue("@maxArea", maxArea.Value);
if (minFloor.HasValue) cmd.Parameters.AddWithValue("@minFloor", minFloor.Value);
if (maxFloor.HasValue) cmd.Parameters.AddWithValue("@maxFloor", maxFloor.Value);
if (cityId.HasValue) cmd.Parameters.AddWithValue("@cityId", cityId.Value);
if (!string.IsNullOrEmpty(addressSearch))
    cmd.Parameters.AddWithValue("@addressSearch", $"%{addressSearch}%");

return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

// ===== ДОБАВИТЬ ФОТОГРАФИЮ КВАРТИРЫ =====
public async Task<int> AddApartmentPhotoAsync(int apartmentId, string url, int order)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

```

```

await SyncPhotoSequenceAsync();

using var cmd = new NpgsqlCommand(@"
INSERT INTO photo_kvartiry (kartochka_kvartiry_id, url_photo, poryadok)
VALUES (@apartmentId, @url, @order)
RETURNING id;", conn);

cmd.Parameters.AddWithValue("@apartmentId", apartmentId);
cmd.Parameters.AddWithValue("@url", url);
cmd.Parameters.AddWithValue("@order", order);

try
{
    var result = await cmd.ExecuteScalarAsync();
    Console.WriteLine($"Фото добавлено: ID={result}, URL={url}");
    return (int)result;
}
catch (Exception ex)
{
    Console.WriteLine($"Ошибка при добавлении фото: {ex.Message}");
    throw;
}
}

private async Task SyncPhotoSequenceAsync()
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    var cmd = new NpgsqlCommand(@"
SELECT setval('photo_kvartiry_id_seq',
              (SELECT COALESCE(MAX(id), 0) + 1 FROM photo_kvartiry),
              false)", conn);

    await cmd.ExecuteNonQueryAsync();
}

```

```

// ===== БРОНИРОВАНИЯ =====

public async Task<int> CheckAvailabilityAsync(int apartmentId, DateTime checkIn, DateTime
checkOut)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(DatabaseQueries.CheckAvailability, conn);
    cmd.Parameters.AddWithValue("@apartment_id", apartmentId);
    cmd.Parameters.AddWithValue("@check_in", checkIn);
    cmd.Parameters.AddWithValue("@check_out", checkOut);
    return Convert.ToInt32(await cmd.ExecuteScalarAsync());
}

// синхронизация айдишников бронирования
private async Task SyncBronirovaniyeSequenceAsync()
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    var cmd = new NpgsqlCommand(@"
SELECT setval('bronirovaniye_id_seq',
              (SELECT COALESCE(MAX(id), 0) + 1 FROM bronirovaniye),
              false)", conn);

    await cmd.ExecuteNonQueryAsync();
}

// Создать бронирование
public async Task<int> CreateBookingAsync(int userId, int apartmentId, DateTime checkIn, DateTime
checkOut)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    await SyncBronirovaniyeSequenceAsync();

    using var cmd = new NpgsqlCommand(@"
INSERT INTO bronirovaniye (arendator_polzovatel_id, kartochka_kvartiry_id,

```



```

        data_zayezda, data_vyyezda,
        status_id, created_at)
VALUES (@userId, @apartmentId, @checkIn, @checkOut,
        (SELECT id FROM status_bronirovaniya WHERE nazvanie = 'Новая'), NOW())
RETURNING id", conn);

cmd.Parameters.AddWithValue("@userId", userId);
cmd.Parameters.AddWithValue("@apartmentId", apartmentId);
cmd.Parameters.AddWithValue("@checkIn", checkIn);
cmd.Parameters.AddWithValue("@checkOut", checkOut);

return (int)await cmd.ExecuteScalarAsync();
}

// Получить бронирования арендодателя
public async Task<NpgsqlDataReader> GetLandlordBookingsAsync(int landlordId)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(@"
SELECT
    b.id,
    b.kartochka_kvartiry_id,
    kk.adres_kvartiry,
    b.arendator_polzovatel_id,
    p.imya || ' ' || p.familiya AS tenant_name,
    p.telefon AS tenant_phone,
    b.data_zayezda,
    b.data_vyyezda,
    s.nazvanie AS status,
    kk.tsena * (b.data_vyyezda - b.data_zayezda) AS total_price,
    b.created_at
FROM bronirovaniye b
JOIN kartochka_kvartiry kk ON b.kartochka_kvartiry_id = kk.id
JOIN polzovatel p ON b.arendator_polzovatel_id = p.id
JOIN status_bronirovaniya s ON b.status_id = s.id
WHERE kk.arendodatel_id = @landlordId
ORDER BY b.data_zayezda ASC", conn);

```

```

        cmd.Parameters.AddWithValue("@landlordId", landlordId);
        return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
    }

    public async Task<int> CancelBookingAsync(int id)
    {
        using var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();

        using var cmd = new NpgsqlCommand(@"
UPDATE bronirovaniye
SET status_id = (SELECT id FROM status_bronirovaniya WHERE nazvanie = 'Отменена')
WHERE id = @id
RETURNING id", conn);
        cmd.Parameters.AddWithValue("@id", id);

        var result = await cmd.ExecuteScalarAsync();
        return result != null ? (int)result : 0;
    }

    // получить бронирования по ID
    public async Task<NpgsqlDataReader> GetBookingByIdAsync(int id)
    {
        var conn = new NpgsqlConnection(_connectionString);
        await conn.OpenAsync();
        var cmd = new NpgsqlCommand(@"
SELECT b.id, b.arendator_polzovatel_id, b.kartochka_kvartiry_id,
       b.data_zayezda, b.data_vyyezda, b.status_id, b.created_at
FROM bronirovaniye b
WHERE b.id = @id", conn);
        cmd.Parameters.AddWithValue("@id", id);
        return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
    }

    // Подтвердить бронирование
    public async Task<int> ConfirmBookingAsync(int id)
    {
        using var conn = new NpgsqlConnection(_connectionString);

```

```

await conn.OpenAsync();

using var cmd = new NpgsqlCommand(@"
UPDATE bronirovaniye
SET status_id = (SELECT id FROM status_bronirovaniya WHERE nazvanie = 'Подтверждена')
WHERE id = @id AND status_id = (SELECT id FROM status_bronirovaniya WHERE nazvanie =
'Новая')
RETURNING id", conn);
cmd.Parameters.AddWithValue("@id", id);

var result = await cmd.ExecuteScalarAsync();
return result != null ? (int)result : 0;
}

// Получить бронирования арендатора
public async Task<NpgsqlDataReader> GetUserBookingsAsync(int userId)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(@"
SELECT
    b.id,
    b.kartochka_kvartiry_id,
    kk.adres_kvartiry,
    kk.tsena,
    b.data_zayezda,
    b.data_vyyezda,
    s.nazvanie AS status,
    kk.tsena * (b.data_vyyezda - b.data_zayezda) AS total_price,
    b.created_at
FROM bronirovaniye b
JOIN kartochka_kvartiry kk ON b.kartochka_kvartiry_id = kk.id
JOIN status_bronirovaniya s ON b.status_id = s.id
WHERE b.arendator_polzovatel_id = @userId
ORDER BY b.data_zayezda ASC", conn);
    cmd.Parameters.AddWithValue("@userId", userId);
    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

```

```

// Проверка доступности квартиры
public async Task<bool> IsApartmentAvailableAsync(int apartmentId, DateTime checkIn, DateTime
checkOut)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    using var cmd = new NpgsqlCommand(@"
SELECT COUNT(*) FROM bronirovaniye
WHERE kartočka_kvartiry_id = @apartmentId
AND status_id != (SELECT id FROM status_bronirovaniya WHERE nazvanie = 'Отменена')
AND data_zayezda < @checkOut
AND data_vyyezda > @checkIn", conn);

    cmd.Parameters.AddWithValue("@apartmentId", apartmentId);
    cmd.Parameters.AddWithValue("@checkIn", checkIn);
    cmd.Parameters.AddWithValue("@checkOut", checkOut);

    var count = Convert.ToInt32(await cmd.ExecuteScalarAsync());
    return count == 0;
}

// Завершить бронирование
public async Task<int> CompleteBookingAsync(int id)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    using var cmd = new NpgsqlCommand(@"
UPDATE bronirovaniye
SET status_id = (SELECT id FROM status_bronirovaniya WHERE nazvanie = 'Завершена')
WHERE id = @id
AND status_id = (SELECT id FROM status_bronirovaniya WHERE nazvanie = 'Подтверждена')
RETURNING id", conn);

    cmd.Parameters.AddWithValue("@id", id);

    var result = await cmd.ExecuteScalarAsync();

```

```

        return result != null ? (int)result : 0;
    }

// ===== СТАТИСТИКА =====

public async Task<int> GetLandlordApartmentsCountAsync(int userId)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(DatabaseQueries.GetLandlordApartmentsCount, conn);
    cmd.Parameters.AddWithValue("@user_id", userId);
    return Convert.ToInt32(await cmd.ExecuteScalarAsync());
}

public async Task<int> GetLandlordActiveBookingsCountAsync(int userId)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(DatabaseQueries.GetLandlordActiveBookingsCount, conn);
    cmd.Parameters.AddWithValue("@user_id", userId);
    return Convert.ToInt32(await cmd.ExecuteScalarAsync());
}

public async Task<decimal> GetLandlordMonthlyIncomeAsync(int userId)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(DatabaseQueries.GetLandlordMonthlyIncome, conn);
    cmd.Parameters.AddWithValue("@user_id", userId);
    return Convert.ToDecimal(await cmd.ExecuteScalarAsync());
}

// ===== ПОИСК =====

public async Task<NpgsqlDataReader> SearchApartmentsAsync(int? cityId, decimal minPrice, decimal
maxPrice, int rooms, DateTime checkIn, DateTime checkOut)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    var cmd = new NpgsqlCommand(DatabaseQueries.SearchApartments, conn);

```

```

cmd.Parameters.AddWithValue("@city_id", cityId ?? (object)DBNull.Value);
cmd.Parameters.AddWithValue("@min_price", minPrice);
cmd.Parameters.AddWithValue("@max_price", maxPrice);
cmd.Parameters.AddWithValue("@rooms", rooms);
cmd.Parameters.AddWithValue("@check_in", checkIn);
cmd.Parameters.AddWithValue("@check_out", checkOut);
return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

```

// ===== СИНХРОНИЗАЦИЯ =====

```

private async Task SyncSequenceAsync(string tableName, string columnName)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    var cmd = new NpgsqlCommand($"@\"
SELECT setval(pg_get_serial_sequence('{tableName}', '{columnName}'),
        COALESCE(MAX({columnName}), 0) + 1,
        false)
FROM {tableName}\"", conn);

    await cmd.ExecuteNonQuery();
}

```

// ===== ОТЗЫВЫ И РЕЙТИНГ =====

// Получить рейтинг квартиры

```

public async Task<double> GetApartmentRatingAsync(int apartmentId)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(@"
SELECT COALESCE(AVG(rk.otsenka), 0)
FROM otzivy o
JOIN reyting_klassifikator rk ON rk.id = o.reyting_klassifikator_id
JOIN bronirovaniye b ON b.id = o.bronirovaniye_id

```

```

WHERE b.kartochka_kvartiry_id = @id AND o.odobreno = B'1'", conn); // ← БЕРУТ с B'1'
cmd.Parameters.AddWithValue("@id", apartmentId);
return Convert.ToDouble(await cmd.ExecuteScalarAsync());
}

// Получить отзывы квартиры (только одобренные)
public async Task<NpgsqlDataReader> GetApartmentReviewsAsync(int apartmentId)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    string sql = @"
SELECT o.id, o.tekst_otzyva, rk.otsenka, o.date,
       p.imya, p.familiya
FROM otzivy o
JOIN reyting_klassifikator rk ON rk.id = o.reyting_klassifikator_id
JOIN bronirovaniye b ON b.id = o.bronirovaniye_id
JOIN polzovatel p ON p.id = b.arendator_polzovatel_id
WHERE b.kartochka_kvartiry_id = @id AND o.odobreno = B'1'
ORDER BY o.date DESC";

    using var cmd = new NpgsqlCommand(sql, conn);
    cmd.Parameters.AddWithValue("@id", apartmentId);

    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

// Добавить отзыв
public async Task<int> AddReviewAsync(int bronirovaniyeId, int ratingId, string text)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    // Синхронизация sequence
    using var syncCmd = new NpgsqlCommand(@"
        SELECT setval('otzivy_id_seq', COALESCE((SELECT MAX(id) FROM otzivy), 0) + 1, false)",
conn);
    await syncCmd.ExecuteNonQueryAsync();

```

```

using var cmd = new NpgsqlCommand(@"
    INSERT INTO otzivy (id, bronirovaniye_id, reyting_klassifikator_id, tekst_otzyva, odobreno)
    VALUES (nextval('otzivy_id_seq'), @b, @r, @t, B'1')
    RETURNING id", conn);

cmd.Parameters.AddWithValue("@b", bronirovaniyeId);
cmd.Parameters.AddWithValue("@r", ratingId);
cmd.Parameters.AddWithValue("@t", text);

return (int)await cmd.ExecuteScalarAsync();
}

// Проверка
public async Task<bool> CanUserReviewAsync(int userId, int bronirovaniyeId)
{
    using var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();
    using var cmd = new NpgsqlCommand(@"
SELECT EXISTS(
    SELECT 1 FROM bronirovaniye b
    WHERE b.id = @bid
    AND b.arendator_polzovatel_id = @uid
    AND b.data_vyyezda < NOW()
    AND NOT EXISTS(SELECT 1 FROM otzivy WHERE bronirovaniye_id = @bid)
)", conn);
    cmd.Parameters.AddWithValue("@bid", bronirovaniyeId);
    cmd.Parameters.AddWithValue("@uid", userId);
    return (bool)await cmd.ExecuteScalarAsync();
}

// получить отзывы арендатора
public async Task<NpgsqlDataReader> GetUserReviewsAsync(int userId)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    string sql = @"
SELECT o.id, b.kartochka_kvartiry_id, kk.adres_kvartiry,

```



```

        rk.otsenka, o.tekst_otzyva, o.date, o.odobreno
    FROM otzivy o
    JOIN reyting_klassifikator rk ON rk.id = o.reyting_klassifikator_id
    JOIN bronirovaniye b ON b.id = o.bronirovaniye_id
    JOIN kartochka_kvartiry kk ON kk.id = b.kartochka_kvartiry_id
    WHERE b.arendator_polzovatel_id = @userId
    ORDER BY o.date DESC";

    using var cmd = new NpgsqlCommand(sql, conn);
    cmd.Parameters.AddWithValue("@userId", userId);

    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}

// Получить отзывы на квартиры арендодателя
public async Task<NpgsqlDataReader> GetLandlordReviewsAsync(int userId)
{
    var conn = new NpgsqlConnection(_connectionString);
    await conn.OpenAsync();

    string sql = @"
    SELECT o.id, kk.id, kk.adres_kvartiry, rk.otsenka, o.tekst_otzyva, o.date,
        p.imya || ' ' || p.familiya as user_name
    FROM otzivy o
    JOIN reyting_klassifikator rk ON rk.id = o.reyting_klassifikator_id
    JOIN bronirovaniye b ON b.id = o.bronirovaniye_id
    JOIN kartochka_kvartiry kk ON kk.id = b.kartochka_kvartiry_id
    JOIN polzovatel p ON p.id = b.arendator_polzovatel_id
    WHERE kk.arendodatel_id = @userId AND o.odobreno = B'1'
    ORDER BY o.date DESC";

    using var cmd = new NpgsqlCommand(sql, conn);
    cmd.Parameters.AddWithValue("@userId", userId);

    return await cmd.ExecuteReaderAsync(CommandBehavior.CloseConnection);
}
}
}

```

```

    Листинг 1.2 — Библиотека интеграционных SQL-запросов DatabaseQueries.cs
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Arenda.DBeaver
{
    public class DatabaseQueries
    {
        public const string GetUserByLogin = @"
        SELECT id, login, parol, email, telefon, imya, familiya
        FROM polzovatel
        WHERE login = @login OR email = @email;";

        public const string CreateUser = @"
        INSERT INTO polzovatel (login, parol, email, telefon, imya, familiya)
        VALUES (@login, @parol, @email, @telefon, @imya, @familiya)
        RETURNING id;";

        public const string GetUserById = @"
        SELECT id, login, parol, email, telefon, imya, familiya, data_rozhdeniya, " as photo
        FROM polzovatel
        WHERE id = @id;";

        public const string UpdateUser = @"
        UPDATE polzovatel
        SET email = @email,
            telefon = @telefon,
            imya = @imya,
            familiya = @familiya,
            data_rozhdeniya = @data_rozhdeniya
        WHERE id = @id;";

        // ===== АВАТАРЫ =====
        public const string GetUserAvatar = @"
        SELECT url_avatar FROM avatary_polzovatelya

```

```
WHERE polzovatel_id = @userId;";
```

```
public const string UpdateUserAvatar = @"  
INSERT INTO avatary_polzovatelya (polzovatel_id, url_avatar, imya_faila, data_obnovleniya)  
VALUES (@userId, @photoUrl, @fileName, NOW())  
ON CONFLICT (polzovatel_id)  
DO UPDATE SET url_avatar = @photoUrl, imya_faila = @fileName, data_obnovleniya = NOW();";
```

```
public const string DeleteUserAvatar = @"  
DELETE FROM avatary_polzovatelya WHERE polzovatel_id = @userId;";
```

```
// ===== КВАРТИРЫ =====
```

```
public const string GetAllApartments = @"  
SELECT  
    kk.id,  
    kk.adres_kvartiry,  
    kk.tsena,  
    kk.ploshad,  
    kk.komnat,  
    g.nazvanie_goroda AS gorod  
FROM kartochka_kvartiry kk  
JOIN gorod g ON kk.gorod_id = g.id  
ORDER BY kk.id DESC;";
```

```
public const string GetApartmentById = @"  
SELECT * FROM kartochka_kvartiry WHERE id = @id;";
```

```
public const string GetApartmentsByLandlord = @"  
SELECT  
    kk.*,  
    g.nazvanie_goroda AS CityName  
FROM kartochka_kvartiry kk  
JOIN gorod g ON kk.gorod_id = g.id  
WHERE kk.arendodatel_id = @userId  
ORDER BY kk.id DESC;";
```

```
public const string CreateApartment = @"  
INSERT INTO kartochka_kvartiry (  

```

```

arendodatel_id, gorod_id, opisaniye_kvartiry, adres_kvartiry,
ploshad, tsena, komnat, etazh, etazhnost, remont,
mebel, tehnika, internet, parkovka, lift, balkon,
zhivotnye_dopustimo, deti_dopustimy, kuriye_dopustimo,
min_srok_v_sutkah, zalog
) VALUES (
    @arendodatel_id, @gorod_id, @opisanie, @adres,
    @ploschad, @tsena, @komnat, @etazh, @etazhnost, @remont,
    @mebel, @tehnika, @internet, @parkovka, @lift, @balkon,
    @zhivotnye_dopustimy, @deti_dopustimy, @kuriye_dopustimo,
    @min_srok, @zalog
) RETURNING id;";

```

```

public const string UpdateApartment = @"
UPDATE kartočka_kvartiry SET
    gorod_id = @gorod_id,
    opisaniye_kvartiry = @opisanie,
    adres_kvartiry = @adres,
    ploshad = @ploschad,
    tsena = @tsena,
    komnat = @komnat,
    etazh = @etazh,
    etazhnost = @etazhnost,
    mebel = @mebel,
    tehnika = @tehnika,
    internet = @internet,
    parkovka = @parkovka,
    lift = @lift,
    balkon = @balkon,
    zhivotnye_dopustimo = @zhivotnye_dopustimy,
    deti_dopustimy = @deti_dopustimy,
    kuriye_dopustimo = @kuriye_dopustimo,
    min_srok_v_sutkah = @min_srok,
    zalog = @zalog
WHERE id = @id
RETURNING id;";

```

```

public const string DeleteApartment = @"

```

```

DELETE FROM kartochka_kvartiry WHERE id = @id;

// ===== БРОНИРОВАНИЯ =====

public const string CheckAvailability = @"
    SELECT COUNT(*) FROM bronirovaniye
    WHERE kartochka_kvartiry_id = @apartment_id
    AND data_zayezda < @check_out
    AND data_vyyezda > @check_in
    AND status != 'Отменена';";

public const string CreateBooking = @"
    INSERT INTO bronirovaniye (
        arendator_polzovatel_id, kartochka_kvartiry_id,
        data_zayezda, data_vyyezda, kolvo_vzroslih, kolvo_detey
    ) VALUES (
        @user_id, @apartment_id, @check_in, @check_out, @adults, @children
    ) RETURNING id;";

public const string GetUserBookings = @"
    SELECT
        b.*,
        kk.adres_kvartiry,
        kk.tsena
    FROM bronirovaniye b
    JOIN kartochka_kvartiry kk ON b.kartochka_kvartiry_id = kk.id
    WHERE b.arendator_polzovatel_id = @user_id
    ORDER BY b.sozdano DESC;";

public const string GetLandlordBookings = @"
    SELECT
        b.*,
        kk.adres_kvartiry,
        p.imya || ' ' || p.familiya AS arendator_name
    FROM bronirovaniye b
    JOIN kartochka_kvartiry kk ON b.kartochka_kvartiry_id = kk.id
    JOIN polzovatel p ON b.arendator_polzovatel_id = p.id
    WHERE kk.arendodatel_id = @user_id
    ORDER BY b.sozdano DESC;";

```

```

public const string CancelBooking = @"
    UPDATE bronirovaniye SET status = 'Отменена'
    WHERE id = @id AND status = 'Новая'
    RETURNING id;";

// ===== СТАТИСТИКА =====
public const string GetLandlordApartmentsCount = @"
    SELECT COUNT(*) FROM kartochka_kvartiry WHERE arendodatel_id = @user_id;";

public const string GetLandlordActiveBookingsCount = @"
    SELECT COUNT(*)
    FROM bronirovaniye b
    JOIN kartochka_kvartiry kk ON b.kartochka_kvartiry_id = kk.id
    WHERE kk.arendodatel_id = @user_id
    AND b.data_vyvezda >= CURRENT_DATE
    AND b.status = 'Новая';";

public const string GetLandlordMonthlyIncome = @"
    SELECT COALESCE(SUM(kk.tsena * (b.data_vyvezda - b.data_zayezda)), 0) AS income
    FROM bronirovaniye b
    JOIN kartochka_kvartiry kk ON b.kartochka_kvartiry_id = kk.id
    WHERE kk.arendodatel_id = @user_id
    AND EXTRACT(MONTH FROM b.data_zayezda) = EXTRACT(MONTH FROM
CURRENT_DATE)
    AND b.status != 'Отменена';";

// ===== ПОИСК =====
public const string SearchApartments = @"
    SELECT
        kk.*,
        g.nazvanie_goroda AS CityName
    FROM kartochka_kvartiry kk
    JOIN gorod g ON kk.gorod_id = g.id
    WHERE
        (g.id = @city_id OR @city_id IS NULL)
        AND kk.tsena BETWEEN @min_price AND @max_price
        AND kk.komnat >= @rooms

```

```
AND NOT EXISTS (  
    SELECT 1 FROM bronirovaniye b  
    WHERE b.kartochka_kvartiry_id = kk.id  
    AND b.data_zayezda < @check_out  
    AND b.data_vyyezda > @check_in  
    AND b.status != 'Отменена'  
)  
ORDER BY kk.tsena;"
```

}

}